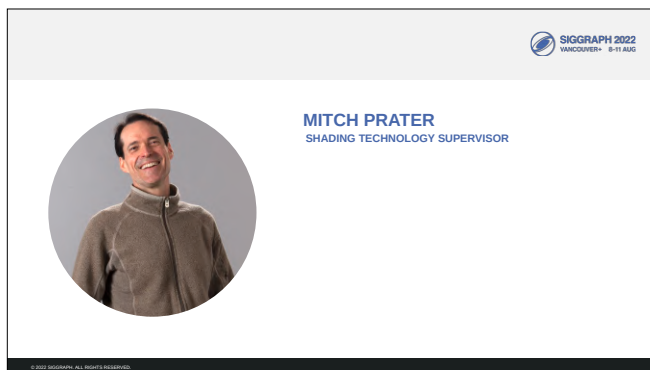
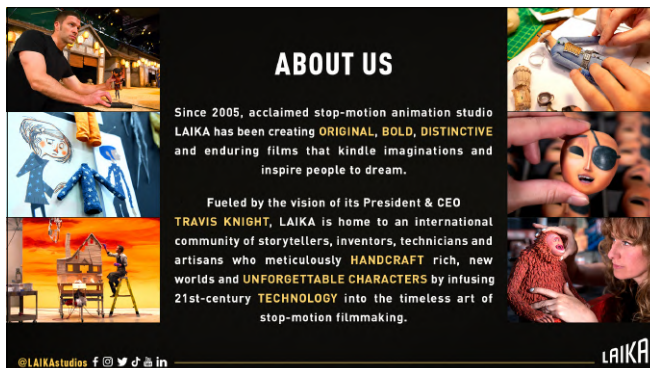




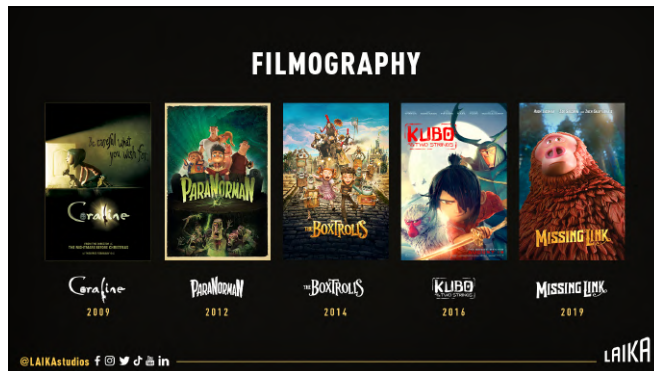
Hello everyone. Thank you for joining me.
This course is about writing OSL Shaders for use in RenderMan.



My name is Mitch Prater and I'm the Shading Technology Supervisor at LAIKA.
My job is focused on the design and implementation of shading and lighting systems for use by the Visual Effects Department.



LAIKA itself is a feature film production studio that is founded on the techniques and traditions of stop-motion.



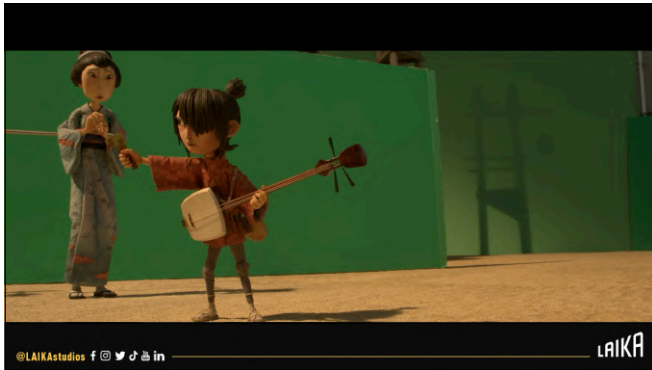
But, we have no fear of developing and using the most cutting-edge technology to expand the range and vision of what we can achieve in our films.



And that's not just hyperbole. We really do create technology and techniques that never existed before across a wide range of fields, including computer graphics and color 3D printing.

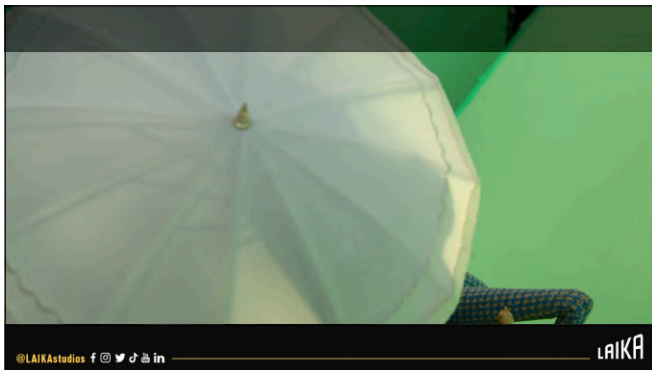


It's also a thoroughly great place to work.



While the stop-motion aspect of our films gets most of the attention, there is a great deal of CG content.

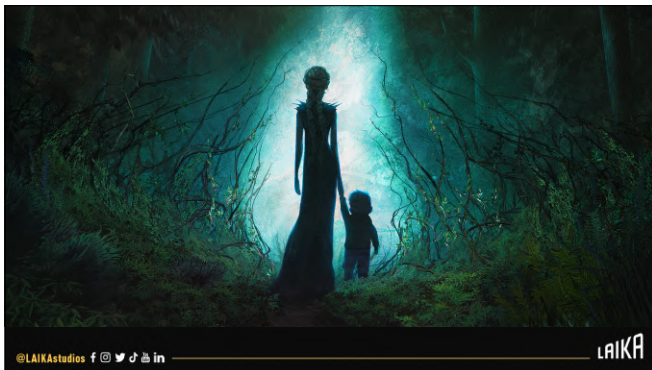
In fact, if you look at the number of shots per film and the amount of CG in those shots,



we're producing the equivalent of a very VFX-intensive live action feature in every film - and possibly more,

since all the secondary characters are completely digital, along with many sets, and all set extensions and backgrounds.

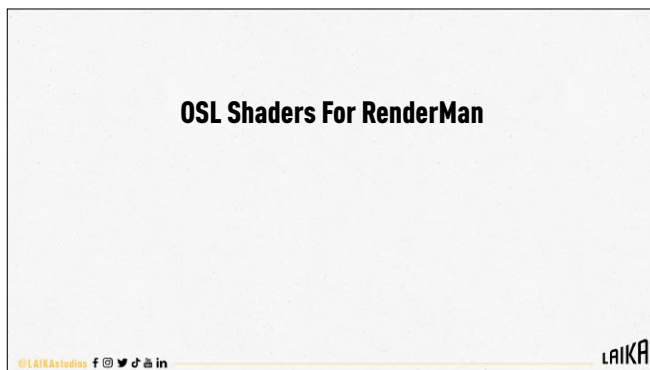
And with each successive film, the amount of pure CG has only increased.



Our current feature production, Wildwood, is going to be even more ambitious.



As for me personally, I've been using computers, and RenderMan in particular, to make pictures for quite a while.



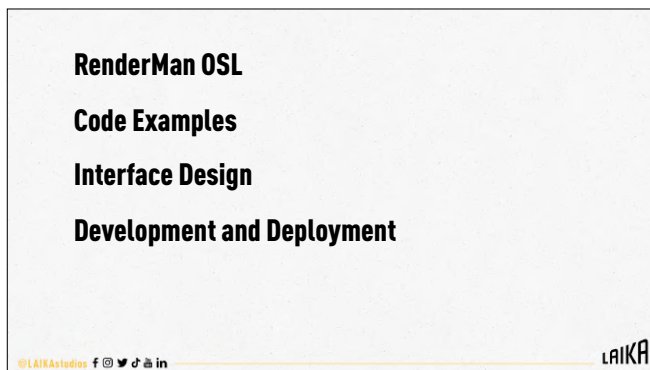
Which brings us to this course.
Shaders make renderings look the way they look.
They represent the intersection between the artistry, and the technology, of rendering.



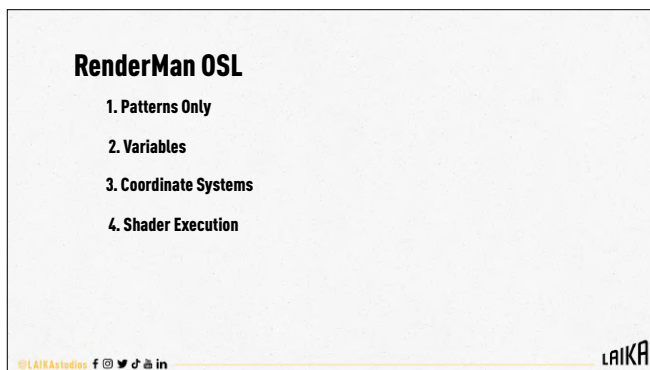
In RenderMan, shaders are written in the Open Shading Language. OSL is a project of the Academy Software Foundation.
The current OSL Specification can be found on the public OSL github repository in src/doc/osl-language-spec.pdf.
Download that document and keep it handy. It is your primary resource for all things relating to the Open Shading Language itself.



RenderMan is Pixar's rendering software ecosystem consisting of a renderer, bridge plugins to several content creation applications, and other supporting software and documentation.



And now for the course, which is divided into four major sections.



From its inception, RenderMan has provided for user-programmable shaders. In fact, the RenderMan Shading Language was part of the original RenderMan Interface Specification. RenderMan's OSL implementation, like other rendering systems that use the Open Shading Language, has its own particular characteristics and usage details. So I'll first cover those.

AUTO

RenderMan OSL

Code Examples

Interface Design

Development and Deployment

@LAIKAstudios f @ t d in

LAIKA

Then, I'll go over some OSL code examples illustrating some of the more fundamental shading tasks.

Code Examples

1. Parameter Conversion
2. Spaces and Transformation
3. Texture Access
4. Procedural Patterns
5. Displacement and Bumping

@LAIKAstudios f @ t d in

LAIKA

AUTO

RenderMan OSL

Code Examples

Interface Design

Development and Deployment

@LAIKAstudios f @ t d in

LAIKA

Interface Design

1. Parameter Naming and Grouping
2. UI Widgets
3. Types & Connections
4. Help Content

After that I'll cover user interface design and the meta-data that's used to create a shader's parameter interface, along with other specifics related to creating a usable production-quality shading node.

RenderMan OSL

Code Examples

Interface Design

Development and Deployment

AUTO

Development and Deployment

1. Source Organization
2. Make System
3. Debugging

And lastly I'll cover the shader development and build environment I use at Laika; which is simple enough for personal use, but has all the capabilities necessary for production deployment, along with some OSL shader debugging techniques.

RenderMan OSL

Code Examples

Interface Design

Development and Deployment

© LAIKAstudios f @ t d in

LAIKA

AUTO

RenderMan OSL

1. Patterns Only

2. Variables

3. Coordinate Systems

4. Shader Execution

© LAIKAstudios f @ t d in

LAIKA

So let's get to it.

RenderMan OSL

1. Patterns Only : *shader* shaders

© LAIKAstudios f @ t d in

LAIKA

First of all, in RenderMan, OSL is used for pattern generation only.

RenderMan OSL

1. Patterns Only : *shader* shaders

No *surface* shaders (no closures or BXDF functions)

No *displacement* shaders (can't write to P)

No *volume* shaders

No *light* shaders

etc...

In future RenderMan versions, OSL might be used for other types of shading; but for now only pattern shaders are allowed.

RenderMan OSL

1. Patterns Only

2. Variables

3. Coordinate Systems

4. Shader Execution

AUTO

RenderMan OSL

2. Variables

Data Types

Compound Types

OSL Global

Built In

Primitive

Options and Attributes

Next are OSL variables.

RenderMan OSL

2. Variables: Data Types

int (also used for bool)
float
color
point, vector, normal
string
matrix

Variables in OSL are comprised of these standard data types. OSL does not have a boolean type; but “int” can be used in its place. Note that ints are 32 bit values.

RenderMan OSL

2. Variables

Data Types
Compound Types
OSL Global
Built In
Primitive
Options and Attributes

AUTO

RenderMan OSL

2. Variables: Compound Types

array one dimensional, 32k max length
struct arrays okay, but no “nesting”

OSL allows the creation of arrays and structs, with some caveats. Arrays are one dimensional as defined by the OSL specification. They can be simple data types, with 32k elements maximum length. Arrays of structs are also okay; and structs can contain members that are arrays - but you can’t have an array of structs that has an array as a member: in other words, no “nesting” of arrays.

RenderMan OSL

2. Variables: Compound Types

array one dimensional, 32k max length
struct arrays okay, but no "nesting"

5.8 Arrays

Arrays of any of the basic types are supported, provided that they are 1D and statically sized, using the usual syntax for C-like languages.

5.9 Structures

Structures are used to group several fields of potentially different types into a single object that can be referred to by name. You may then use the structure type name to declare structure variables as you would for any of the built-in types.

© LAIKAstudios f @ t d in

LAIKA

You can find all the details about how to declare and use arrays and structs in the OSL Specification Sections 5.8 and 5.9.

RenderMan OSL

2. Variables

Data Types
Compound Types
OSL Global
Built In
Primitive
Options and Attributes

© LAIKAstudios f @ t d in

LAIKA

AUTO

RenderMan OSL

2. Variables: OSL Global

P N Ng I u v

© LAIKAstudios f @ t d in

LAIKA

Now we'll get into the meat of shading, which is creating variation over the surface of an object.

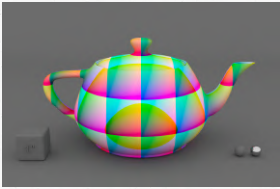
To do that, we need to know where we are on the surface.

To start, there are pre-defined, or OSL "global" variables. These Global Variables contain values that change depending on exactly what point on the surface is currently being shaded by the renderer.

RenderMan OSL

2. Variables: OSL Global

P N Ng I u v



```
shader global_P
{
    output color_P = color(0.0)
}
{
    point P_object;
    P_object = transform( "common", "object", P );
    color_P = P_object - floor( P_object );
}
```

@LAIKAstudios f @ t d a in

LAIKA

To access an OSL global variable, you simply use it.

P is the three-dimensional point on the surface of an object. In RenderMan, P is read-only: as I mentioned before, no displacement can be performed by writing to it. RenderMan currently uses a c++ plugin to do displacement. However, if displacement is used, the value of P will be changed by that displacement to reflect the new surface location after the displacement has been applied. So in your OSL code, P will be the displaced surface location. In this image, its values are ramped in a sawtooth pattern by subtracting the floor(), or integer portion, of the point from itself.

RenderMan OSL

2. Variables: OSL Global

P N Ng I u v



```
shader global_N
{
    output color_N = color(0.0)
}
{
    normal N_object;
    N_object = transform( "common", "object", N );
    N_object = normalize( N_object );
    color_N = abs( N_object );
}
```

@LAIKAstudios f @ t d a in

LAIKA

N and Ng are surface normals. Normals describe the surface orientation. N is typically called the “shading” normal.

Ng is the geometric surface normal. Like P, both are also influenced by displacement.

RenderMan OSL

2. Variables: OSL Global

P N Ng I u v



```
shader global_I
{
    output color_I = color(0.0)
}
{
    vector I_object;
    I_object = transform( "common", "object", I );
    color_I = abs( normalize( I_object ) );
}
```

@LAIKAstudios f @ t d a in

LAIKA

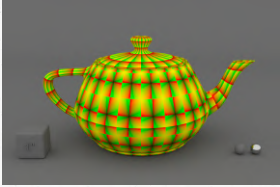
I is the “eye” vector, as in the vector pointing from the observer, or eye - in this case the camera - to the surface.

If the object were being shaded as a result of a secondary, bounced ray, the I vector would originate from that location and point toward the shaded object.

RenderMan OSL

2. Variables: OSL Global

P N Ng I **u** **v**



```
shader global_uv
{
    output color_uv = color(0.0)
}
{
    color_uv = color( u, v, 0 );
}
```

© LAIKAstudios f @ t d in

LAIKA

u and v come from the geometry itself and its inherent surface parameterization. Specifically, u and v are not the texture mapping “UV” coordinates; those are actually contained in a primitive variable called “st”. But we’ll get to primvars in a minute after we talk about the RenderMan Builtin Variables.

RenderMan OSL

2. Variables

Data Types
Compound Types
OSL Global
Built In
Primitive
Options and Attributes

© LAIKAstudios f @ t d in

LAIKA

AUTO

RenderMan OSL

2. Variables: Built In

© LAIKAstudios f @ t d in

LAIKA

Builtin variables are like OSL global variables, but they are defined by RenderMan itself and not OSL.

RenderMan OSL

2. Variables: Built In

```
getattribute( "builtin", "name", var );
```

Builtin variables are accessed with the somewhat general-purpose `getattribute()` function.

The *name* of the builtin variable is provided as a string, as well as a previously declared variable in which to place it. The variable must be of the correct type for the named builtin value, or the call will fail and the variable will be left unchanged.

RenderMan OSL

2. Variables: Built In

```
getattribute( "builtin", "name", var );
```

```
point Po;
if( getattribute( "builtin", "Po", Po ))
{
    printf( "Got Po." );
}
else
{
    error( "Po not accessible." );
}
```

`getattribute()` provides a return value of 1 to indicate success, and returns 0 if the call fails for any reason.

RenderMan OSL

2. Variables: Built In

```
getattribute( "builtin", "name", var );
```

manwiki.pixar.com/display/REN24/PxrVariable

There are a total of 41 builtin variables defined by RenderMan,
most of which are accessible with the `getattribute()` function.

`/opt/pixar/RenderManProServer/include/RixShading.h`

All of the RenderMan builtin variables are accessed using the `getattribute()` function.

A list of *many* of these is shown in the PxrVariable shading node's documentation.

In total, there are currently 41 builtin variables defined by RenderMan. These are all listed in the `RixShading.h` file in the RenderManProServer installation's include directory.

RenderMan OSL

2. Variables: Built In

```
getattribute( "builtin", "name", var );
```

<i>Po</i>	Original, un-displaced surface point.
<i>Non</i>	Original, un-displaced surface normal.
<i>Tn</i>	Surface tangent vector.
<i>curvature</i>	Surface curvature float.
<i>Id</i>	Object identifier int.
<i>Id2</i>	Second object identifier (only katana) int.

@LAIKAstudios f @ t d in

LAIKA

These are some of the most commonly used builtin variables provided by RenderMan.

When displacement is used, the OSL Global P and N values are updated to match the displaced surface. But Po and Non retain the surface's original, un-displaced values.

Tn is the surface tangent vector. When texture mapping "UV's" are present, Tn points in the "U" direction.

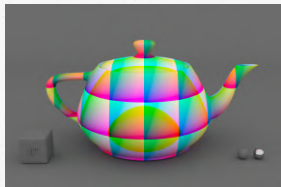
The curvature values are computed analytically from the geometry.

The Id values uniquely identify each object, so no two will have the same Id values.

RenderMan OSL

2. Variables: Built In

Po



```
shader builtin_Po
(
  output color_Po = color(0.0)
)
{
  point Po;
  getattribute( "builtin", "Po", Po );
  Po = transform( "common", "object", Po );
  color_P = Po - floor( Po );
}
```

@LAIKAstudios f @ t d in

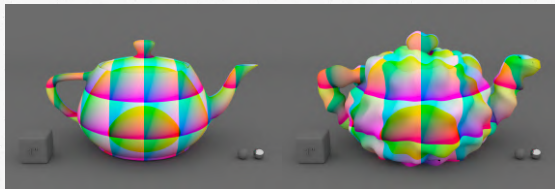
LAIKA

Here's what Po looks like. It's identical to P on an un-displaced surface. But since Po represents the un-displaced surface point, if the surface is displaced, ...

RenderMan OSL

2. Variables: Built In

Po



@LAIKAstudios f @ t d in

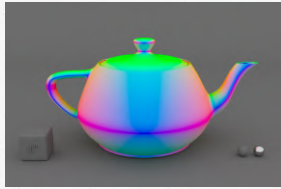
LAIKA

...the shaded coloration doesn't change - it "sticks" to the surface as it's stretched and distorted by displacement.

RenderMan OS!

2. Variables: Built In

Non



```
shader builtin_Non
{
    output color_Non = color(0.0)
}
{
    normal Non;
    getattribute( "builtin", "Non", Non );
    Non = transform( "common", "object", Non );
    Non = normalize( Non );
    color_Non = abs( Non );
}
```

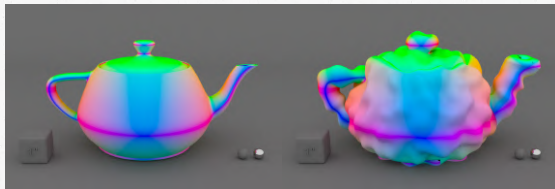
LAIKA

Non is similar.
When the surface is displaced ...

RenderMan OS!

2. Variables: Built In

Non



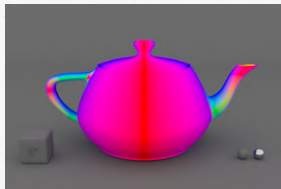
LAIKA

...Non will remain pointing in the original surface's normal direction.

RenderMan OS!

2. Variables: Built In

Tn



```
shader builtin_Tn
{
    output color_Tn = color(0.0)
}
{
    vector Tn;
    getattribute( "builtin", "Tn", Tn );
    Tn = transform( "common", "object", Tn );
    Tn = normalize( Tn );
    color_Tn = abs( Tn );
}
```

LAIKA

Tn is interesting. Since it's a surface tangent vector, it can be used to define the direction of anisotropy for an anisotropic specular response. But the interesting thing is that when the geometry has texture mapping "UV's" - Tn becomes the gradient of the U direction; so Tn will point in a direction that follows the U component's path over the surface. In this way, Tn can be used to create shading that follows the flow of the UV's; which can be pretty useful.

RenderMan OSL

2. Variables: Built In curvature



```
shader builtin_curvature
{
    output color_curv = color(0.0)
}
{
    float curv;
    getattribute( "builtin", "curvature", curv );
    color_curv = color( curv );
}
```

© LAIKAstudios f @ t d a in

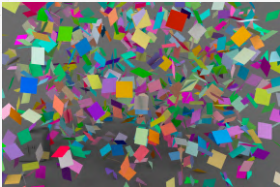
LAIKA

curvature is a measure of the overall surface curvature.

Positive values represent convex areas and Negative values represent concave curvature.

RenderMan OSL

2. Variables: Built In Id, Id2



```
shader builtin_Id
{
    output color_Id = color(0.0)
}
{
    int Id, Id2;
    getattribute( "builtin", "Id", Id );
    getattribute( "builtin", "Id2", Id2 );

    point Idpoint = point( Id, Id2, 0 );
    color_Id = noise( "cell", Idpoint );
}
```

© LAIKAstudios f @ t d a in

LAIKA

Id and Id2 contain a unique integer value for every object in the scene. Using these, a single shader can automatically produce a different result for every object on which it's used.

Note that Id2 is only set when using the RenderMan for Katana bridge application. In all others, Id2 is 0.

RenderMan OSL

2. Variables

- Data Types
- Compound Types
- OSL Global
- Built In
- Primitive
- Options and Attributes

© LAIKAstudios f @ t d a in

LAIKA

AUTO

RenderMan OSL

2. Variables: Primitive

© LAIKAstudios f @ t d in

LAIKA

Next are the Primitive Variables.

Primitive Variables are defined as additional data associated with the geometry: that is, they are additional components of the geometry, and are part of the geometric primitive's definition. Hence the name: primitive variable.

RenderMan OSL

2. Variables: Primitive

```
getattribute( "primvar", "name", var );
```

© LAIKAstudios f @ t d in

LAIKA

Primvars are also accessed with the `getattribute()` function.

As with builtin variables, you provide the name of the primitive variable, as well as an actual variable in which to place its value. Again, the variable must be the correct type to receive the primitive variable's value, or the call will fail and the variable will be left unchanged.

RenderMan OSL

2. Variables: Primitive

```
getattribute( "primvar", "name", var );
```

point, vector, and normal primitive variable types are provided in "common" space. To get the original numerical values, they must be transformed to "object" space:

```
var = transform( "common", "object", var );
```

© LAIKAstudios f @ t d in

LAIKA

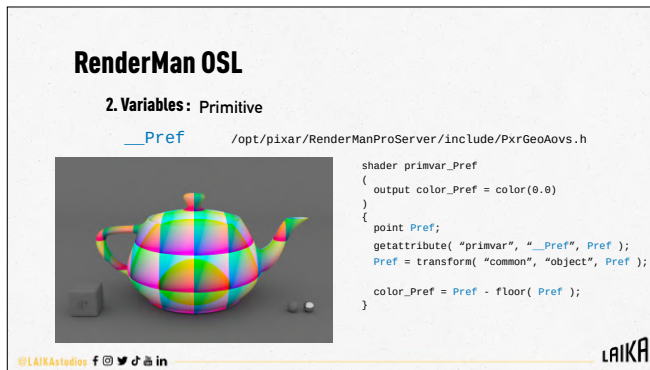
int, float, and color primvar data types are unaltered when they are accessed with the `getattribute()` function: so their values remain exactly as they were specified on the geometry.

However, point, vector, and normal data types are provided in "common" space, just as the OSL global variables are.

"common" space, however, is different than the space in which the values were originally specified, so their values inside the shader won't be the same as those on the geometry.

To get those original values back - which is almost always what you want since those are the values you attached to the geometry in the first place -

you must transform the value returned by `getattribute()` from “common” space back to “object” space, since “object” space is the space in which the geometric object, and so its primitive variables, were defined.



This is what it looks like in shader code. Note the name of the “Pref” primvar has a “__” prefix.

This is a Pixar-ism that dates back to the earliest days of their primitive variable implementation.

For some reason it was decided that in addition to an “r e f” suffix, primitive variables should also have a “__” prefix.

At any rate, this naming convention has been around since the beginning and is codified into RenderMan.

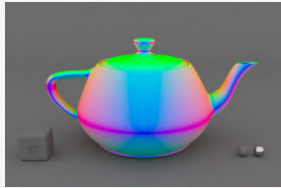
There exists a list of such primitive variables with this naming convention that, when present, will automatically be written to a correspondingly named Arbitrary Output Variable, or AOV, display channel.

This list of so-called “Geo AOV’s” is detailed in the RenderManProServer’s `PxrGeoAovs.h` file. This list also includes the Builtin variables.

RenderMan OS!

2. Variables: Primitive

__Nref /opt/pixar/RenderManProServer/include/PxrGeoAovs.h



```
shader primvar_Nref
{
    output color_Nref = color(0.0)
}
{
    normal Nref;
    getattribute( "primvar", "__Nref", Nref );
    Nref = transform( "common", "object", Nref );
    color_Nref = abs( Nref );
}
```

LAIKA

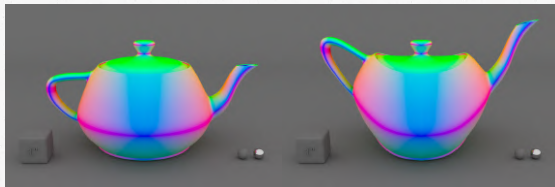
Nref is used for surface normal values that presumably have been normalized before being attached to the geometry; so transforming Nref back into its original “object” space will produce the original, normalized data; which is why normalizing it after the transform() call is not necessary.

Now, why do we need Pref and Nref when we have the builtin variables Po and Non?

RenderMan OS!

2. Variables: Primitive

__Nref



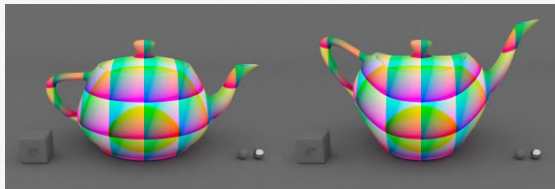
LAIKA

Recall that Po and Non are unaffected by displacement. Well, Pref and Nref, in addition to being immune to displacement, are also immune to deformation.

RenderMan OS!

2. Variables: Primitive

__Pref



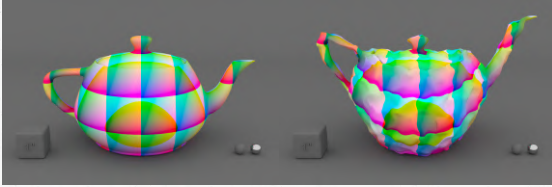
LAIKA

So even though an object is deformed by animation, using Pref and Nref for shading causes the shaded results to stick to the Pref and Nref defined surface ...

RenderMan OSL

2. Variables: Primitive

`__Pref`



© LAIKAstudios f @ t d in

LAIKA

...no matter what the original P and N values are doing - before or after deformation or displacement.

This is why primitive variables exist: to provide a fixed reference that always sticks to the surface.

RenderMan OSL

2. Variables: Primitive

```
float s=0.0 [[int lockgeom=0]]
float t=0.0 [[int lockgeom=0]]
```

Declared as shader parameters. Contain the texture "UV"s if they exist, or the geometry's parametric u and v values if they do not.

© LAIKAstudios f @ t d in

LAIKA

Which brings us to the "st" primitive variable. s and t are the texture Mapping "UV"s", as they're called, used by 3d paint applications; but they are in fact stored in the "st" primitive variable.

Unlike other primitive variables, the "st" primvar is accessed as two separate shader parameters rather than using pre-defined "s" and "t" global variables, or the `getattribute()` function.

This particular mechanism works whether the "st" data is defined as a two-dimensional float array as... (shown)

RenderMan OSL

2. Variables: Primitive

```
float s=0.0 [[int lockgeom=0]]
float t=0.0 [[int lockgeom=0]]
```

Declared as shader parameters. Contain the texture "UV"s if they exist, or the geometry's parametric u and v values if they do not.

```
"facevarying float[2] st" [0.505190849 0.548759103 0.527539104 0.548759103 ...]
```

© LAIKAstudios f @ t d in

LAIKA

...shown in this RenderMan Interface example, or as two separate "s" and "t" float primvars.

Also, if the "st" primvars aren't present, this mechanism will automatically assign the geometry's parametric "uv" space to the s and t parameters.

RenderMan OSL

2. Variables: Primitive

s t



```
shader primvar_st
{
    float s=0.0 [[int lockgeom=0]],
    float t=0.0 [[int lockgeom=0]],
    output color_st = color(0.0)
}
{
    color_st = color( s, t, 0 );
}
```

@LAIKAstudios f @ t d in

LAIKA

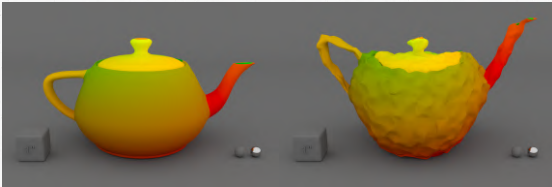
This is what it looks like in shader code.

Again, because the “st” values are defined as a primvar, the object can be displaced and deformed...

RenderMan OSL

2. Variables: Primitive

s t



@LAIKAstudios f @ t d in

LAIKA

...and any shading based on the s and t parameters will remain stuck to the surface.

RenderMan OSL

2. Variables

- Data Types
- Compound Types
- OSL Global
- Built In
- Primitive
- Options and Attributes

@LAIKAstudios f @ t d in

LAIKA

AUTO

RenderMan OSL

2. Variables: Options and Attributes

LAIKAstudios f @ t d in

LAIKA

Lastly, we have Option and Attribute variables.

RenderMan OSL

2. Variables: Options and Attributes

rmanwiki.pixar.com/display/REN24/Renderer+Options

rmanwiki.pixar.com/display/REN24/Instance+Attributes

rmanwiki.pixar.com/display/REN24/Primitive+Variables

Examples:

identifier:id	dice:micropolygonlength
Ri:Orientation	grouping:membership
trace:bias	Ri:FormatResolution

LAIKAstudios f @ t d in

LAIKA

There are quite a number of these, and they're covered in these links to the relevant RenderMan documentation.

The documentation also lists what Pixar is now calling "Prototype Attributes" on a page that's currently entitled "Primitive Variables" for some reason - but they're still just attributes.

These all have two-part names consisting of category-colon-name.

The difference between Options and Attributes is that Options are additional data values that control what RenderMan does, or provide information about, a single rendering as a whole.

Whereas Attributes are additional data that specify characteristics of an object, whether that is a piece of geometry, a camera, a light source, or any other abstract "object" used in RenderMan.

Attributes are hierarchically scoped in the RenderMan scene description, and they apply to all entities within that scope.

RenderMan OSL

2. Variables: Options and Attributes

```
getattribute( "category:name", var );
```

Again, these are accessed with the `getattribute()` function, just like the Builtin and Primitive variables.

RenderMan OSL

2. Variables: Options and Attributes

```
getattribute( "user:name", var );
```

When category = user, specifying the category is optional:

```
getattribute( "name", var );
```

In addition to the RenderMan-defined Options and Attributes, you can create your own “user” category attributes and associate them with objects in your scene.

When accessing a user attribute in the shader, the category-colon portion of the specifier can be left off and you can simply specify the name.

RenderMan OSL

1. Patterns Only

2. Variables

3. Coordinate Systems

4. Shader Execution

AUTO

RenderMan OSL

3. Coordinate Systems (a.k.a. Spaces)

©LAIKAstudios f @ t d a in

LAIKA

Now we'll cover the Coordinate Systems, which are also known as "Spaces".

RenderMan OSL

3. Coordinate Systems (a.k.a. Spaces)

```
"common"      manwiki.pixar.com/display/REN24/Coordinate+Systems
"object"
"world"
"camera"
"screen"
"NDC"
"raster"
"name"
```

©LAIKAstudios f @ t d a in

LAIKA

We talked a bit about this when transforming primitive variables from "common" space back into their original "object" space, but there are a few other useful spaces available in OSL.

RenderMan OSL

3. Coordinate Systems

```
"common"
"object"
"world"
"camera"
"screen"
"NDC"
"raster"
"name"

shader coordinate_system
(
    output color_space = color(0.0)
)
{
    point Pspace;
    Pspace = transform( "common", "space", P );
    color_space = Pspace - floor( Pspace );
}
```

©LAIKAstudios f @ t d a in

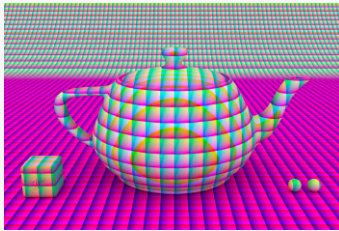
LAIKA

This shader code uses the transform() function to transform the "common" space P value into the desired coordinate system and generates a color from it.

RenderMan OSL

3. Coordinate Systems

"common"
"object"
"world"
"camera"
"screen"
"NDC"
"raster"
"name"



LAIKA

"common" is the default coordinate system in which all point, vector, and normal values are provided, whether they're global, builtin, or primitive variables.

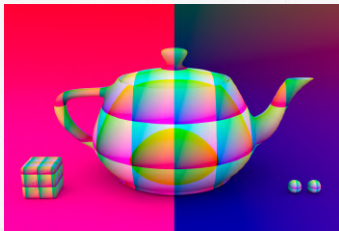
It is an arbitrary reference space where the internal displacement and rendering computations take place.

You should never make an assumption about what this space is, which is why we always transform variables into another coordinate system before using them to generate a pattern.

RenderMan OSL

3. Coordinate Systems

"common"
"object"
"world"
"camera"
"screen"
"NDC"
"raster"
"name"



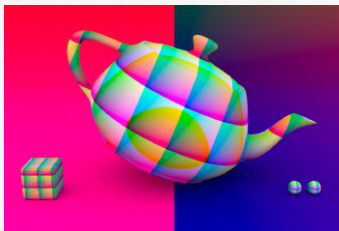
LAIKA

"object" space is relative to each individual object in the scene. Any transformations applied to the object are also applied to the object's "object" space.

RenderMan OSL

3. Coordinate Systems

"common"
"object"
"world"
"camera"
"screen"
"NDC"
"raster"
"name"



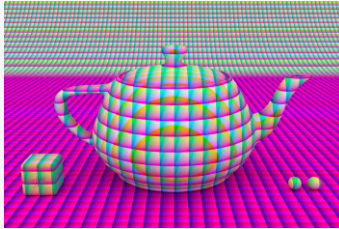
LAIKA

In this case, the teapot has been scaled up in size relative to the reference cube and spheres in the scene, and has been further transformed. The background object has also been scaled up quite a bit relative to these.

RenderMan OSL

3. Coordinate Systems

"common"
"object"
"world"
"camera"
"screen"
"NDC"
"raster"
"name"



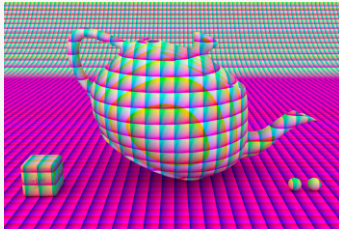
LAIKA

"world" space is the "scene" coordinate system. It is rigid and uniform throughout the entire world.

RenderMan OSL

3. Coordinate Systems

"common"
"object"
"world"
"camera"
"screen"
"NDC"
"raster"
"name"



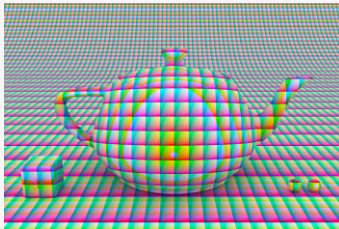
LAIKA

As objects are moved through the scene, they will "swim" through the "world" space.

RenderMan OSL

3. Coordinate Systems

"common"
"object"
"world"
"camera"
"screen"
"NDC"
"raster"
"name"



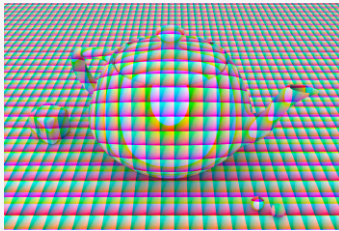
LAIKA

"camera" space represents the three-dimensional transformation applied to the camera that's used to view the scene.

RenderMan OSL

3. Coordinate Systems

"common"
"object"
"world"
"camera"
"screen"
"NDC"
"raster"
"name"



@LAIKAstudios f @ t d in

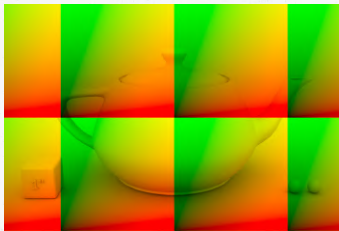
LAIKA

So as the camera moves, this space will move through the scene and will not be fixed to any of the objects in it.

RenderMan OSL

3. Coordinate Systems

"common"
"object"
"world"
"camera"
"screen"
"NDC"
"raster"
"name"



@LAIKAstudios f @ t d in

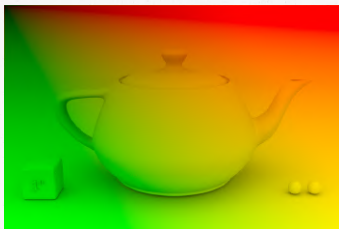
LAIKA

"screen" space is relative to the camera's image plane. It is two-dimensional, and its origin is at the center of the camera's view, with positive values to the right and up, and negative values going to the left and down.

RenderMan Overview

3. Coordinate Systems

"common"
"object"
"world"
"camera"
"screen"
"NDC"
"raster"
"name"



@LAIKAstudios f @ t d in

LAIKA

"NDC" stands for normalized device coordinates, and represents a 0 to 1 ranged coordinate system that describes the output image rectangle. (0,0) is in the upper left corner, and (1,1) is in the lower right corner.

RenderMan OSL

3. Coordinate Systems

"common"
"object"
"world"
"camera"
"screen"
"NDC"
"raster"
"name"



© LAIKAstudios f @ t d in

LAIKA

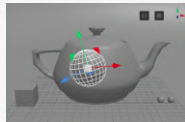
Then we have “raster” space. This is the two-dimensional pixel coordinate system. That is, each pixel is one unit in “raster” space.

Here, each pixel contains the same red-green ramp we’ve seen previously; but since the pixels are - the size of a pixel - the resulting image becomes uniformly colored by the average value of the red-green pattern.

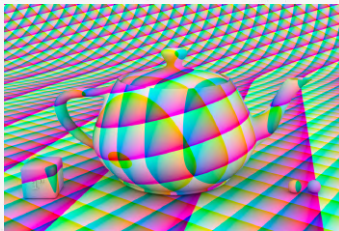
Like “NDC” space, the origin of each pixel is in its upper left corner.

RenderMan OSL

3. Coordinate Systems



"NDC"
"raster"
"name"



© LAIKAstudios f @ t d in

LAIKA

In addition to the pre-defined coordinate systems, you can also define your own “named” coordinate systems.

These are arbitrary spaces that you can add to your scene, and are referenced by the name you’ve given them.

These are created using a RenderMan Bridge application.

RenderMan OSL

1. Patterns Only
2. Variables
3. Coordinate Systems
4. Shader Execution

© LAIKAstudios f @ t d in

LAIKA

AUTO

RenderMan OSL

4. Shader Execution

Displacement v.s. Bxdf
JIT Optimization
Strings and Filenames
Auto Differentiation

© LAIKAstudios f @ t d in

LAIKA

Here I'll cover topics related to how OSL shaders are executed in RenderMan.

RenderMan OSL

4. Shader Execution : Displacement v.s. Bxdf



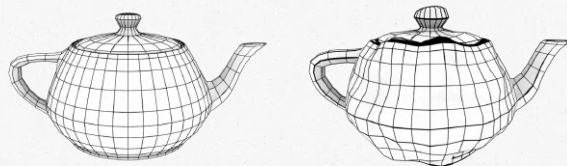
© LAIKAstudios f @ t d in

LAIKA

The first thing to keep in mind is that displacement and the “shading and lighting” portions of rendering are done independently of each other.

RenderMan OSL

4. Shader Execution : Displacement v.s. Bxdf



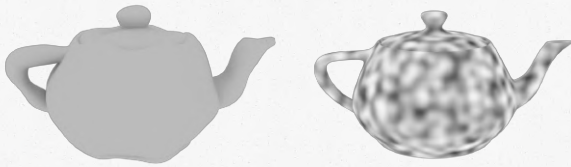
© LAIKAstudios f @ t d in

LAIKA

The displacement process is run first. It's here that the geometry is “diced” into micro-polygons, which are typically about the size of a pixel, and only the displacement portion of the shading graph is compiled and executed in order to move those micro-polygons into their displaced positions. This process is done prior to any image generation.

RenderMan OSL

4. Shader Execution : Displacement v.s. Bxdf



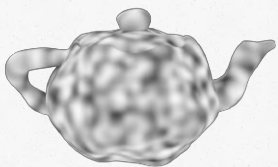
© LAIKAstudios f @ t d in

LAIKA

The other thing to keep in mind is that the displacement process defines “common” space differently than the “shading and lighting”, or Bxdf execution portion of rendering that’s used during image generation. These teapots are using the exact same noise pattern based on P. But since P was used in its raw “common” space, the displaced pattern and the “shaded” pattern are clearly different.

RenderMan OSL

4. Shader Execution : Displacement v.s. Bxdf



```
shader common_space
{
    output RGB = color(0.0);
    output point Pdisp = point(0.0);
}
float height = noise( P );
Pdisp = P + N*height;
RGB = color( height );
}
```

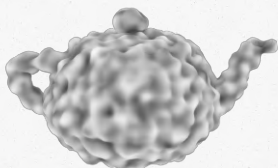
© LAIKAstudios f @ t d in

LAIKA

If this shader is used to do both, this will be the result. Clearly the shaded pattern and the displaced pattern aren’t the same.

RenderMan OSL

4. Shader Execution : Displacement v.s. Bxdf



```
shader world_space
{
    output RGB = color(0.0);
    output point Pdisp = point(0.0);
}
point Pw = transform( "common", "world", P );
float height = noise( Pw );
Pdisp = P + N*height;
RGB = color( height );
}
```

© LAIKAstudios f @ t d in

LAIKA

Instead, if you first transform P from “common” space into a “known” coordinate system, in this case “world” space, then the displaced and shaded noise patterns will be using the same space.

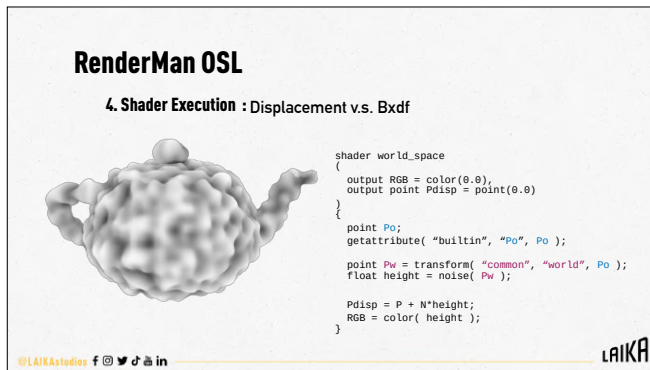
However, if you look carefully, you might notice something slightly odd about this result: the value of the gray color doesn’t exactly match the displaced height.

To understand why, remember that displacement and bxdf shading are executed by two different processes.

The displacement process runs first, and creates the new surface that is then

used by the bxdf shading process.

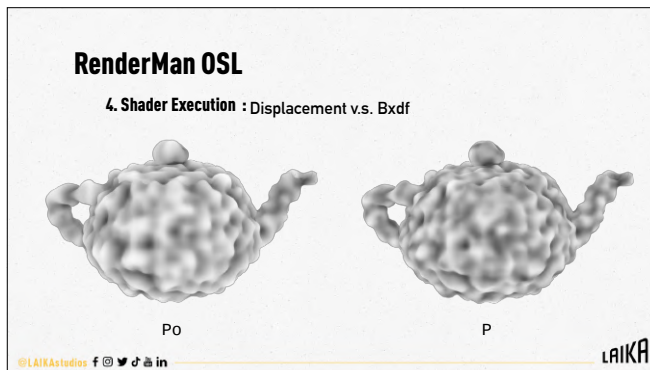
Since the displacement changes the surface, the OSL Global variable P will have a new and different value when the bxdf shading process is run; so its noise() result is different than the displaced noise() result.



In order for the displacement value of height and the bxdf shaded value of height to be identical, Po must be used instead of P.

Recall that Po is the original, un-displaced surface point, so using it in your shader instead of P ensures the displacement *and* the bxdf shading produce the same result.

Using Po, we now have the displaced height and the height coloration sticking to the original surface, and therefore they match each other.



Here, you can clearly see the difference. While the displacements are identical, the bxdf shading is not.

In summary, not only must you always transform your pattern generation variables away from “common” and into a “known” space, you must also choose the correct variable depending on what kind of bxdf result you want.

AUTO

RenderMan OSL

4. Shader Execution

Displacement v.s. Bxdf

JIT Optimization

Strings and Filenames

Auto Differentiation

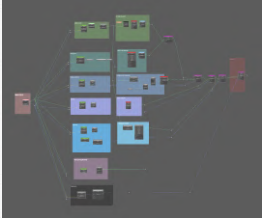
LAIKAstudios

f @ t d i n

LAIKA

RenderMan OSL

4. Shader Execution : JIT Optimization



```
1: float3 P = vec(0.0, 0.0, 0.0);
2: float3 N = vec(0.0, 0.0, 0.0);
3: float3 T = vec(0.0, 0.0, 0.0);
4: float3 B = vec(0.0, 0.0, 0.0);
5: float3 R = vec(0.0, 0.0, 0.0);
6: float3 G = vec(0.0, 0.0, 0.0);
7: float3 B = vec(0.0, 0.0, 0.0);
8: float3 A = vec(0.0, 0.0, 0.0);
9: float3 S = vec(0.0, 0.0, 0.0);
10: float3 L = vec(0.0, 0.0, 0.0);
11: float3 M = vec(0.0, 0.0, 0.0);
12: float3 V = vec(0.0, 0.0, 0.0);
13: float3 W = vec(0.0, 0.0, 0.0);
14: float3 X = vec(0.0, 0.0, 0.0);
15: float3 Y = vec(0.0, 0.0, 0.0);
16: float3 Z = vec(0.0, 0.0, 0.0);
17: float3 U = vec(0.0, 0.0, 0.0);
18: float3 V = vec(0.0, 0.0, 0.0);
19: float3 W = vec(0.0, 0.0, 0.0);
20: float3 X = vec(0.0, 0.0, 0.0);
21: float3 Y = vec(0.0, 0.0, 0.0);
22: float3 Z = vec(0.0, 0.0, 0.0);
23: float3 U = vec(0.0, 0.0, 0.0);
24: float3 V = vec(0.0, 0.0, 0.0);
25: float3 W = vec(0.0, 0.0, 0.0);
26: float3 X = vec(0.0, 0.0, 0.0);
27: float3 Y = vec(0.0, 0.0, 0.0);
28: float3 Z = vec(0.0, 0.0, 0.0);
29: float3 U = vec(0.0, 0.0, 0.0);
30: float3 V = vec(0.0, 0.0, 0.0);
31: float3 W = vec(0.0, 0.0, 0.0);
32: float3 X = vec(0.0, 0.0, 0.0);
33: float3 Y = vec(0.0, 0.0, 0.0);
34: float3 Z = vec(0.0, 0.0, 0.0);
35: float3 U = vec(0.0, 0.0, 0.0);
36: float3 V = vec(0.0, 0.0, 0.0);
37: float3 W = vec(0.0, 0.0, 0.0);
38: float3 X = vec(0.0, 0.0, 0.0);
39: float3 Y = vec(0.0, 0.0, 0.0);
40: float3 Z = vec(0.0, 0.0, 0.0);
41: float3 U = vec(0.0, 0.0, 0.0);
42: float3 V = vec(0.0, 0.0, 0.0);
43: float3 W = vec(0.0, 0.0, 0.0);
44: float3 X = vec(0.0, 0.0, 0.0);
45: float3 Y = vec(0.0, 0.0, 0.0);
46: float3 Z = vec(0.0, 0.0, 0.0);
47: float3 U = vec(0.0, 0.0, 0.0);
48: float3 V = vec(0.0, 0.0, 0.0);
49: float3 W = vec(0.0, 0.0, 0.0);
50: float3 X = vec(0.0, 0.0, 0.0);
51: float3 Y = vec(0.0, 0.0, 0.0);
52: float3 Z = vec(0.0, 0.0, 0.0);
53: float3 U = vec(0.0, 0.0, 0.0);
54: float3 V = vec(0.0, 0.0, 0.0);
55: float3 W = vec(0.0, 0.0, 0.0);
56: float3 X = vec(0.0, 0.0, 0.0);
57: float3 Y = vec(0.0, 0.0, 0.0);
58: float3 Z = vec(0.0, 0.0, 0.0);
59: float3 U = vec(0.0, 0.0, 0.0);
60: float3 V = vec(0.0, 0.0, 0.0);
61: float3 W = vec(0.0, 0.0, 0.0);
62: float3 X = vec(0.0, 0.0, 0.0);
63: float3 Y = vec(0.0, 0.0, 0.0);
64: float3 Z = vec(0.0, 0.0, 0.0);
65: float3 U = vec(0.0, 0.0, 0.0);
66: float3 V = vec(0.0, 0.0, 0.0);
67: float3 W = vec(0.0, 0.0, 0.0);
68: float3 X = vec(0.0, 0.0, 0.0);
69: float3 Y = vec(0.0, 0.0, 0.0);
70: float3 Z = vec(0.0, 0.0, 0.0);
71: float3 U = vec(0.0, 0.0, 0.0);
72: float3 V = vec(0.0, 0.0, 0.0);
73: float3 W = vec(0.0, 0.0, 0.0);
74: float3 X = vec(0.0, 0.0, 0.0);
75: float3 Y = vec(0.0, 0.0, 0.0);
76: float3 Z = vec(0.0, 0.0, 0.0);
77: float3 U = vec(0.0, 0.0, 0.0);
78: float3 V = vec(0.0, 0.0, 0.0);
79: float3 W = vec(0.0, 0.0, 0.0);
80: float3 X = vec(0.0, 0.0, 0.0);
81: float3 Y = vec(0.0, 0.0, 0.0);
82: float3 Z = vec(0.0, 0.0, 0.0);
83: float3 U = vec(0.0, 0.0, 0.0);
84: float3 V = vec(0.0, 0.0, 0.0);
85: float3 W = vec(0.0, 0.0, 0.0);
86: float3 X = vec(0.0, 0.0, 0.0);
87: float3 Y = vec(0.0, 0.0, 0.0);
88: float3 Z = vec(0.0, 0.0, 0.0);
89: float3 U = vec(0.0, 0.0, 0.0);
90: float3 V = vec(0.0, 0.0, 0.0);
91: float3 W = vec(0.0, 0.0, 0.0);
92: float3 X = vec(0.0, 0.0, 0.0);
93: float3 Y = vec(0.0, 0.0, 0.0);
94: float3 Z = vec(0.0, 0.0, 0.0);
95: float3 U = vec(0.0, 0.0, 0.0);
96: float3 V = vec(0.0, 0.0, 0.0);
97: float3 W = vec(0.0, 0.0, 0.0);
98: float3 X = vec(0.0, 0.0, 0.0);
99: float3 Y = vec(0.0, 0.0, 0.0);
100: float3 Z = vec(0.0, 0.0, 0.0);
```

LAIKAstudios

f @ t d i n

LAIKA

Prior to rendering, the entire shading network graph and all its parameter interconnections and settings are “just in time”, or JIT compiled, into an optimized shader executable. These shading network optimizations are done in two passes: once for displacement, and once for bxdx shading. So a single shading network is compiled into two separate executables, just prior to displacement and image generation. The JIT process distills the entire shading graph and all its interconnections and parameter and variable settings into an optimized executable so that any constants expressed by the shading network or its shader code are folded into a constant value at compile time, and any dead code is removed, along with many other optimizations. This results in a shader that is as efficient to execute as possible when displacing and rendering.

RenderMan OSL

4. Shader Execution : JIT Optimization

```
shader optimizer_example
(
    int BlackOrWhite = 0,
    output color RGB = color(0.0)
)
{
    if( BlackOrWhite )
    {
        RGB = color(1.0);
    }
    else
    {
        RGB = color(0.0);
    }
}

Source
```

```
shader optimizer_example
(
    int BlackOrWhite = 0,
    output color RGB = color(0.0)
)
{
    if( BlackOrWhite )
    {
        RGB = color(1.0);
    }
    else
    {
        RGB = color(0.0);
    }
}

Optimized
```

© LAIKAstudios f @ t d in

LAIKA

This is just an illustration of one type of optimization: this isn't what the compiled code actually looks like. But if the if() statement's condition is based on a constant, the compiler can eliminate it entirely.

RenderMan OSL

4. Shader Execution : JIT Optimization

```
shader optimizer_example
(
    int BlackOrWhite = 0,
    output color RGB = color(0.0)
)
{
    if( BlackOrWhite )
    {
        RGB = color(1.0);
    }
    else
    {
        RGB = color(0.0);
    }
}

Source
```

```
shader optimizer_example
(
    int BlackOrWhite = 0,
    output color RGB = color(0.0)
)
{
    if( BlackOrWhite )
    {
        RGB = color(1.0);
    }
    else
    {
        RGB = color(0.0);
    }
}

Optimized
```

© LAIKAstudios f @ t d in

LAIKA

And if the BlackOrWhite parameter is set to a constant value of 1 by the shading network, the compiler will know that too, and instead eliminate the other half of the if() statement.

There are many other details about what the optimizer can do and how it works; but as one who programs OSL shaders, you simply need to be aware, that this, and many other types of optimization, take place based on the entire shading network's interconnections and parameter settings, and whether those values vary over the surface, or are constant.

RenderMan OSL

4. Shader Execution

- Displacement v.s. Bxdf
- JIT Optimization
- Strings and Filenames
- Auto Differentiation

© LAIKAstudios f @ t d in

LAIKA

AUTO

RenderMan OS!

4. Shader Execution : Strings and Filenames

© LAIKAstudios f @ t d in

LAIKA

Strings of course are also part of the JIT optimization process.

RenderMan OS!

4. Shader Execution : Strings and Filenames

String operations are allowed, but a string must JIT-evaluate to a constant.

```
shader texture_Read
(
    string RepoPath = "",
    string AssetName = "",
    string DataType = "diffuse_color",
    string Extension = ".tex",
    output RGB = color(0.0)
)
{
    string file = concat( RepoPath, "/", AssetName, "/" DataType, Extension );
    RGB = texture( file, - );
}
```

© LAIKAstudios f @ t d in

LAIKA

So you can have string parameters, variables, string parameter connections between nodes, and you can use string functions to concat them, make substitutions, and use other string operations - just so long as they end up being constant values prior to rendering. In this case, we're concatenating several string parameter values to build up a single texture file path name.

RenderMan OS!

4. Shader Execution : Strings and Filenames

Filename strings can employ Token Substitution for dynamic string construction.

© LAIKAstudios f @ t d in

LAIKA

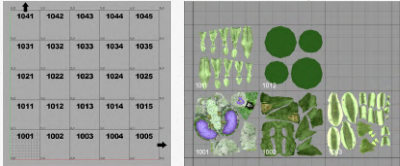
However, there are times when a string value does need to change over the surface of an object, such as accessing the correct texture file to apply to a UV indexed surface patch.

For dynamically specifying filenames, Token Substitution is used.

RenderMan OSL

4. Shader Execution : Strings and Filenames

Filename strings can employ Token Substitution for dynamic string construction.



@LAIKAstudios f @ t d in

LAIKA

Token Substitution allows you to insert sub-strings into a Filename that are changed at run time.

For these Mari-style UV Maps with UDIM indexing, a Filename Token is used to dynamically insert the proper numerical value into the Filename's String so that the proper texture is used on the surface patches matching its UDIM index.

RenderMan OSL

4. Shader Execution : Strings and Filenames

Filename strings can employ Token Substitution for dynamic string construction.

Style	Token	Filename String	Accessed File(s)
Mari	<udim> or <UDIM>	diffuse.<udim>.tex	diffuse.1001.tex, diffuse.1002.tex, etc.
Zbrush	<U> and <V>	diffuse.u<u>_v<v>.tex	diffuse.u0_v0.tex, diffuse.u1_v0.tex, etc.
Mudbox	<U> and <V>	diffuse.u<u>_v<v>.tex	diffuse.u1_v1.tex, diffuse.u2_v1.tex, etc.
User Attribute or Primitive Variable	<primstr:name>	<primstr:rough_file>.tex	roughness.tex

@LAIKAstudios f @ t d in

LAIKA

There are various types of file token to support different paint applications. There are Mari-style UDIM file sets, Zbrush style atlas indices, or Mudbox style file naming.

In addition to texture atlas indexing, string-valued User Attributes and Primitive Variables can be substituted into a file's string using the "primstr" token.

RenderMan OSL

4. Shader Execution : Strings and Filenames

```
shader texture_Read
{
    string RepoPath = "/laika/textures",
    string AssetName = "<primstr:asset_name>",
    string Filename = "diffuse_color.<udim>.tex",
    output RGB = color(0.0)
}
{
    string file = concat( RepoPath, "/", AssetName, "/" Filename );
    RGB = texture( file, - );
}
```

Accessed Texture Files :

/laika/textures/painted_object/diffuse_color.1001.tex
/laika/textures/painted_object/diffuse_color.1002.tex
etc.

@LAIKAstudios f @ t d in

LAIKA

In this example, there is a string User Attribute called "asset-underscore-name" whose value is "painted_object". This object was painted using the Mari 3d paint application, which saves a number of different texture files that contain the paint for different UV-indexed surface patches. Using File Tokens, the texture() function will access the correct texture files for each surface patch, based on the filenames Mari saved its paint in.

This is all done efficiently by the texture() function's implementation, and avoids the use of any dynamically created strings in the shader.

AUTO

RenderMan OSL

4. Shader Execution

- Displacement v.s. Bxdf
- JIT Optimization
- Strings and Filenames
- Auto Differentiation

©LAIKAstudios f @ t d in

LAIKA

These functions use the differential of their argument to produce their result.

RenderMan OSL

4. Shader Execution : Auto Differentiation

```
Dx() Dy() Dz()  
area()  
filterwidth()  
calculatenormal()  
aastep()
```

©LAIKAstudios f @ t d in

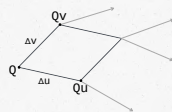
LAIKA

OSL does not use the change in the differentiated variable from one micro-polygon to the next - in other words, the finite difference - to approximate the variable's derivative.

RenderMan OSL

4. Shader Execution : Auto Differentiation

```
Dx() Dy() Dz()  
area()  
filterwidth()  
calculatenormal()  
aastep()
```



$$\begin{aligned}dQ_u &= Q_u - Q \\dQ_v &= Q_v - Q\end{aligned}$$

©LAIKAstudios f @ t d in

LAIKA

RenderMan OSL

4. Shader Execution : Auto Differentiation

```
Dx() Dy() Dz()  
area()  
filterwidth()  
calculatenormal()  
aastep()
```



Q = shader code.
dQ = shader code transformed
to compute Q's derivative.

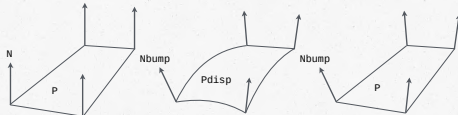
LAIKAstudios f @ t d in

LAIKA

Instead it uses Auto Differentiation, which computes the derivative based on the OSL code that generates the variable's value. While this is actually more correct than using finite differences, it can be quite involved, and represents a fair amount of computational gymnastics. As a result, only values that are functions of certain Variables can have their derivatives computed.

RenderMan OSL

4. Shader Execution : Auto Differentiation



```
point Pobjct = transform( "common", "object", P );  
float height = noise( Pobjct );
```

```
Pdisp = P + N*height;  
Nbump = calculatenormal( Pdisp );
```

LAIKAstudios f @ t d in

LAIKA

In particular, values that can trace their origin back to the global variable P can be used with the derivative functions.

Among other things, this is how a “bumped” shading normal, Nbump, is generated by the calculatenormal() function so we can shade with the bumped shading normal and the un-displaced surface.

This gives the shaded surface the appearance that it is displaced without actually displacing it.

Other than P, I don't have a definitive list of which variables can be differentiated.

In particular, N currently can't, so using the derivative of N to create some measure of the surface curvature is not allowed.

RenderMan OSL

4. Shader Execution : Auto Differentiation



Also, second and other higher-order derivatives are not allowed: you can't take the derivative of a derivative.

RenderMan OSL

Code Examples

Interface Design

Development and Deployment

That's it for the RenderMan specifics of OSL.

Code Examples

1. Parameter Conversion
2. Spaces and Transformation
3. Texture Access
4. Procedural Patterns
5. Displacement and Bumping

In this section, we'll look at some real-world code examples.

Code Examples

1. Parameter Conversion

Simple Types

Artistic to Physical

@LAIKAstudios f @ t d a in

LAIKA

To start we'll look at converting parameters from one type to another.

Code Examples

1. Parameter Conversion : Simple Types

@LAIKAstudios f @ t d a in

LAIKA

Simple type conversions are fairly trivial, but also very useful.

Code Examples

1. Parameter Conversion : Simple Types

```
shader convert_FloatToColor
{
    float Gain = 1.0,
    float R = 1.0,
    float G = 1.0,
    float B = 1.0,
    output color Color = color( 0.0 )
}
Color = Gain * color( R, G, B );
```

@LAIKAstudios f @ t d a in

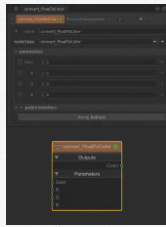
LAIKA

This is a fully functioning shading node that converts float values to a color. This also shows all the major sections of a shader. The first line contains the OSL shader type, which is simply "shader", which denotes that this is a pattern shader; followed by its name. Next is the parameter section in parenthesis. Here, there are four input parameters, and one output parameter. Then comes the shader code body that uses the input parameter values to compute the output parameter's value. And that's it. All shaders, no matter how complex, will follow this same structure.

Code Examples

1. Parameter Conversion : Simple Types

```
shader convert_FloatToColor
{
    float Gain = 1.0,
    float R = 1.0,
    float G = 1.0,
    float B = 1.0,
    output color Color = color( 0.0 )
}
Color = Gain * color( R, G, B );
```



LAIKAstudios f @ t d in

LAIKA

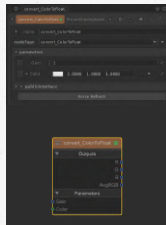
When this shader is compiled and used in a RenderMan Bridge application, it will appear as a shading node that can be used in a shader graph. Its input parameters are presented in a user interface where you can set their values directly.

The node is also presented in the shader node graph window where the input and output parameters can potentially be connected to other shading nodes. Specifically, its output must be connected to another shading node or it won't be used.

Code Examples

1. Parameter Conversion : Simple Types

```
shader convert_ColorToFloat
{
    float Gain = 1.0,
    color Color = color(1.0),
    output float R = 0.0,
    output float G = 0.0,
    output float B = 0.0,
    output float AvgRGB = 0.0
}
{
    R = Gain * Color.r;
    G = Gain * Color.g;
    B = Gain * Color.b;
    AvgRGB = ( R + G + B ) / 3.0;
}
```



LAIKAstudios f @ t d in

LAIKA

This shader does the opposite of the previous example: it converts a color into its component float values.

It also provides the average of all the color component values.

Note that the struct-like notation for the Color parameter is being used to access its separate components.

Color, point, vector, and normal variables can use either this struct-like notation using dot-rgb or dot-xyz, or array subscript notation can be used for accessing the individual components.

You'll also note that this node has more than one output parameter.

The fact is, a pattern shader doesn't need any input parameters at all.

But it must have at least one output parameter; otherwise, it's not going to be connectable to anything, and therefore won't be of any use.

Code Examples

1. Parameter Conversion

Simple Types

Artistic to Physical

@LAIKAstudios f @ t d in

LAIKA

A more complex type of conversion is to transform a set of Artistic parameters into a Physical set of parameters.

Code Examples

1. Parameter Conversion : Artistic to Physical

```
shader convert_BMTtoSPE
{
    color BaseColor = color(0.5),
    float Metallic = 0.9,
    color DielectricFace = color(0.04),
    color DielectricEdge = color(1.0),
    color MetallicBede = color(0.0),
    output color Diffuse = color(0.0),
    output color SpecularFace = color(0.0),
    output color SpecularEdge = color(0.0)
}
Diffuse = mix( BaseColor, MetallicAlbedo, Metallic );
SpecularFace = mix( DielectricFace, BaseColor, Metallic );
SpecularEdge = mix( DielectricEdge, BaseColor, Metallic );
```

@LAIKAstudios f @ t d in

LAIKA

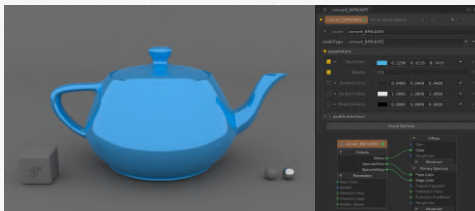
This conversion isn't exactly Artistic to Physical, but it is a frequently used conversion that takes Base Color and Metallic values and converts them to Diffuse and Specular response colors.

This conversion has been around for many years.

The difference in this implementation is that the control parameters, shown in the middle of the parameter declaration section, are exposed to the user so they can have the ultimate control to fine-tune the results.

Code Examples

1. Parameter Conversion : Artistic to Physical



@LAIKAstudios f @ t d in

LAIKA

Here's a semi-metallic blue material. Note the outputs are connected to a PxrSurface bxdf shader.

Code Examples

1. Parameter Conversion : Artistic to Physical

```
shader convert_SF2BK
(
    color SpecularFace = color(0.0),
    color SpecularEdge = color(1.0),
    output color n = color(0.0),
    output color k = color(0.0)
)
```

LAIKAstudios f @ t d in

LAIKA

The last conversion is quite a bit more complex, so I'll break it down by parts. Here are the shader's parameters.

It uses the “Artistic” controls specifying the Specular Face and Specular Edge colors, and converts them into a Refractive Index, n , and the Extinction Coefficient, k .

The combination of n and k is called the “complex refractive index”, which is used by physical specular response models for conductive materials.

Code Examples

1. Parameter Conversion : Artistic to Physical

```
{
    float square(float x) { return x*x; }

    // Schlick, D. 2014
    // Artistic Friendly Metallic Fresnel
    // Journal of Computer Graphics Techniques Vol. 3, No. 4
    //
    float f0f90_to_eta(float f0, float f90)
    {
        float eta = (1.0-f0)/(1.0+f0);
        float sqrt_f0 = sqrt(f0);
        float eta_max = (1.0+sqrt_f0)/(1.0-sqrt_f0);
        eta_max = select(1.0, eta_max, isfinite(eta_max));
        return mix(eta_max, eta, f90);
    }

    float f0eta_to_k(float f0, float eta)
    {
        float nr = f0*eta*eta*eta*eta*eta*eta*eta*eta*eta*eta;
        float k = sqrt(nr/(1.0-f0));
        return select(1.0, k, isfinite(k));
    }
}
```

LAIKAstudios f @ t d in

LAIKA

In the body of the shader code, we start with a comment about where this conversion method came from, and then define some functions that will be used later.

The square() function does the obvious thing.

The next function converts the SpecularFace value, which is $f0$, and the SpecularEdge value, which is $f90$, into a corresponding index of refraction, η .

This function uses a couple of useful OSL functions - select() and isfinite() - in order to ensure the maximum index of refraction, η_{max} , contains a useful value.

select() acts like the C language's conditional expression, or question mark, operator.

select() exists specifically to avoid run-time branching with an if() statement in the executable; which might slow down execution.

The last function converts the SpecularFace value, and index of refraction, into the extinction coefficient, k .

Code Examples

1. Parameter Conversion : Artistic to Physical

```
color f0f90_to_etal color f0, color f90 )
{
    return color(
        f0f90_to_etal f0[0], f90[0] ),
        f0f90_to_etal f0[1], f90[1] ),
        f0f90_to_etal f0[2], f90[2] )
}

color f0eta_to_kf color f0, color eta )
{
    return color(
        f0eta_to_kf f0[0], eta[0] ),
        f0eta_to_kf f0[1], eta[1] ),
        f0eta_to_kf f0[2], eta[2] )
}

color eta = f0f90_to_etal(SpecularFace, SpecularEdge );
k = f0eta_to_kf(SpecularFace, eta );
n = color(1.0,0.0,0.0);
n = select( color(1.0,0.0,0.0), n, color( isfinite(n[0]), isfinite(n[1]), isfinite(n[2]) ) );
```

@LAIKAsudios f @ t w i n

LAIKA

The next couple of functions take the previously defined float functions and uses them to define new functions that operate on color values.

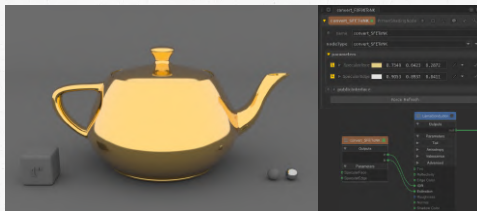
Here you can see the array index notation being used to access the rgb components of the color values instead of the struct-like notation you saw previously. Either one is fine.

Finally comes the actual conversion of the input parameters to the output parameters.

Note, on the last line, that the select() function works on color variable types, as well as point, vector, and normal types, in which case it tests and selects each component individually. isfinite() unfortunately doesn't work on tuple types, so three separate calls are needed to make up its color value for the select() function.

Code Examples

1. Parameter Conversion : Artistic to Physical



@LAIKAsudios f @ t w i n

LAIKA

And here's the result using some representative face and edge colors for gold, and the LamaConductor bxdf.

AUTO

Code Examples

1. Parameter Conversion
2. Spaces and Transformation
3. Texture Access
4. Procedural Patterns
5. Displacement and Bumping

@LAIKAstudios f @ t d a in

LAIKA

Next come Spaces and Transformation.

Code Examples

2. Spaces and Transformation
- 2D
 - 3D

@LAIKAstudios f @ t d a in

LAIKA

We'll start with a simple 2D node.

It has an integer parameter named Coordinates that lets us select where the 2D space comes from,

and a PrimitiveVariable string parameter that lets us specify the name of a primvar if we want to explicitly choose one.

The result goes into the output point parameter.

Then we have the s and t parameters that will be automatically set to the Texture Mapping "st" primvar, if it's present.

Code Examples

2. Spaces and Transformation : 2D

```
shader space_2D
{
    int Coordinates = 0, // @TextureUV 1:PrimVar 2:Geo uv
    string PrimitiveVariable = "st",
    output point Out = point(0,0),
    float s = 0, // [[ int lockgeom = 0 ]],
    float t = 0, // [[ int lockgeom = 0 ]],
}
```

@LAIKAstudios f @ t d a in

LAIKA

Code Examples

2. Spaces and Transformation : 2D

```
{
    // Get at space: the Texture UV's.
    Out = point(s, t, 0);
    // Primitive Variable.
    if (1 == Coordinates)
    {
        float primvar[2];
        if (getattribute("primvar", PrimitiveVariable, primvar))
        {
            Out.s = primvar[0];
            Out.y = primvar[1];
        }
    }
    // Geometric uv.
    else if (2 == Coordinates)
    {
        Out.s = u;
        Out.y = v;
    }
}
```

@LAIKAstudios f @ t d a in

LAIKA

Out is first set to the s and t values.

Then we check if the Coordinates parameter is set to select either of the alternatives.

If there's an explicit PrimVar we want to use, getattribute() will access that, or we'll set the Output to the Geometric u and v values instead.

And that's it.

Code Examples

2. Spaces and Transformation :

2D

3D

@LAIKAstudios f @ t d a in

LAIKA

AUTO

Code Examples

2. Spaces and Transformation : 3D

```
shader space_3D
{
    int DisplacedP = 0;
    string Coordinates = "Object", // Set to "Primitive Variable" to use it instead.
    string PrimitiveVariable = "_Pref",
    string SizeSpace = "world";
    output point Out = point(0,0)
}
```

@LAIKAstudios f @ t d a in

LAIKA

Now we'll look at a shader that produces a 3-dimensional space.

DisplacedP, when set, bases the result on the displaced surface instead of the original surface.

The Coordinates parameter specifies what coordinate space we want to use. If instead we want to use a primvar, then we set Coordinates to "Primitive Variable".

That will cause the shader to access the primvar specified by the PrimitiveVariable parameter.

SizeSpace lets us use any Coordinate space we want the resulting point to stick to, but scales its size to match the size space.

Code Examples

2. Spaces and Transformation : 3D

```
// Referencing P is required for any down-stream auto-differentiation.
Out = P;

// Replace with un-displaced P?
if (DisplacedP) getattribute( "builtin", "P", Out );

// Check for "Primitive Variable" Coordinates.
if (PrimitiveVariable == Coordinates)
{
    if (getattribute( "primvar", PrimitiveVariable, Out ))
    {
        warning(
            "A point-type Primitive Variable = \"%s\" wasn't found. "
            "Using \"Object\" Coordinates instead.",
            PrimitiveVariable
        );
    }
}

// Restore the original primvar values.
// or start with "Object" Coordinates.
Out = transform( "common", "Object", Out );
```

LAIKAstudios f t d in

LAIKA

We start by setting the output to P, which is the displaced, “common” space surface point.

Then it's replaced with the “original” P if DisplacedP is zero, which is typically what we want.

Next, if the Coordinates have been set to "Primitive Variable", we'll access a primvar instead.

At this point, whether Out is from a primvar or a surface point, the transform() call, using string arguments for the spaces, will restore the primvar to its original values, or transform the surface point from "common" space to “object” space: this is correct for both.

Code Examples

2. Spaces and Transformation : 3D

```
if (PrimitiveVariable != Coordinates)
{
    matrix object_to_coord;
    if (getmatrix( "Object", Coordinates, object_to_coord ))
    {
        // Transform to Coordinates.
        Out = transform( object_to_coord, Out );

        // Apply SizeSpace scale.
        matrix coord_to_size;
        if (getmatrix( Coordinates, SizeSpace, coord_to_size ))
        {
            Out *= pow( abs( determinant( coord_to_size ) ), 0.33333333 );
        }
        else
        {
            warning( "Size Space = \"%s\" wasn't found.", SizeSpace );
        }
    }
    else
    {
        warning( "Coordinates = \"%s\" wasn't found.", Coordinates );
    }
}
```

LAIKAstudios f t d in

LAIKA

With Out now in “object” space, we what to transform it to our desired Coordinate space, **if** it did not come from a primvar.

If so, we start by checking if the Coordinates parameter specifies a usable space by checking the return value of getmatrix(), which puts the transformation matrix into its argument.

If that succeeds, we then use the transform() function with the matrix argument to transform Out to the desired Coordinates.

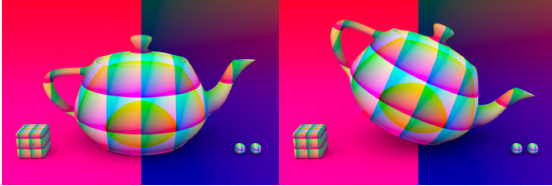
Then we're going to apply the SizeSpace scaling. This allows us to make the output stick to the Coordinates we want, but scale it so any patterning based on Out will be consistent with the SizeSpace we want.

It does this by computing the ratio of the volumes between the two spaces, which is done by taking the cube-root of the absolute value of the determinant of the matrix that transforms from one space to the other, and scales the output by that amount.

But why go to all this trouble of scaling the result to SizeSpace?

Code Examples

2. Spaces and Transformation : 3D



@LAIKAstudios f @ t d in

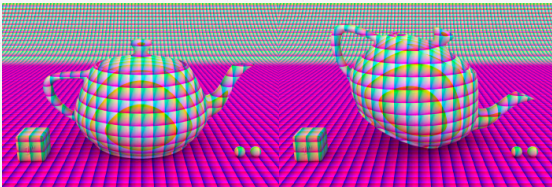
LAIKA

Recall that “object” space follows any transformations that are applied to the individual objects.

That’s good thing when you’re moving objects around in the scene - you want any shaded patterning to be attached to them and to move with them.

Code Examples

2. Spaces and Transformation : 3D



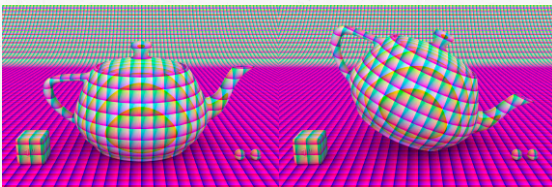
@LAIKAstudios f @ t d in

LAIKA

Also recall that “world” space, while being uniformly sized throughout the scene, does not follow the objects’ transformations.

Code Examples

2. Spaces and Transformation : 3D



@LAIKAstudios f @ t d in

LAIKA

Using SizeSpace scaling, the resulting value, while still sticking to the objects, is now scaled to a consistent size.

That’s the whole purpose of the SizeSpace parameter: to make the space uniformly scaled, while still sticking to the desired Coordinates.

AUTO

Code Examples

1. Parameter Conversion
2. Spaces and Transformation
- 3. Texture Access**
4. Procedural Patterns
5. Displacement and Bumping

@LAIKAstudios f @ t d in

LAIKA

Code Examples

- 3. Texture Access**
- Filename Tokens
 - t Inversion
 - Texture Info

@LAIKAstudios f @ t d in

LAIKA

Now to a texture access example, which will cover these RenderMan-specific topics.

Code Examples

3. Texture Access

```
shader texture_read
{
    string File = "";
    point Space = point(0,0),
    float s = 0.0 [[ int lockgeom = 0 ]],
    float t = 0.0 [[ int lockgeom = 0 ]],
    output color RGB = color(0.0),
    output float R = 0.0,
    output float G = 0.0,
    output float B = 0.0,
    output float A = 0.0
}
```

@LAIKAstudios f @ t d in

LAIKA

Starting with the shader's parameters, we have a string File parameter that will contain the texture filename, a point Space parameter that will be used to map the texture onto the surface, and then the 2D Texture Mapping UV primvar parameters. We'll use those if the Space parameter is not connected to something. Finally we have the output parameters.

Code Examples

3. Texture Access

```
{
    // Texture mapping space.
    point xy = point(s,t,0);
    if( isconnected( Space )) xy = Space;
```

@LAIKAstudios f @ t d in

LAIKA

In the body of the shader, we start by setting the xy point to the raw texture space.

Code Examples

3. Texture Access : Filename Tokens

```
{
    // Texture mapping space.
    point xy = point(s,t,0);
    if( isconnected( Space )) xy = Space;
    // Check for indexing tokens in the File string.
    int udim = regex_search( File, "~udim" ) || regex_search( File, "~uDim" );
    int uv = regex_search( File, "~uv" ) && regex_search( File, "~vuv" );
    int U = regex_search( File, "~u" ) && regex_search( File, "~v" );
    int uv_index = udim || uv || U;
```

@LAIKAstudios f @ t d in

LAIKA

Then we search the File parameter string to see if it contains any of the filename uv tokens.

Code Examples

3. Texture Access : t Inversion

```
{
    // Texture mapping space.
    point xy = point(s,t,0);
    if( isconnected( Space )) xy = Space;
    // Check for indexing tokens in the File string.
    int udim = regex_search( File, "~udim" ) || regex_search( File, "~uDim" );
    int uv = regex_search( File, "~uv" ) && regex_search( File, "~vuv" );
    int U = regex_search( File, "~u" ) && regex_search( File, "~v" );
    int uv_index = udim || uv || U;
    // Flip the orientation for RenderMan .tex file access.
    // This requires re-orienting each indexed UV region separately.
    if( uv_index )
    {
        float index = floor(xy.y);
        xy.y = 1 - (xy.y - index) + index;
    }
    else
    {
        xy.y = 1 - xy.y;
    }
}
```

@LAIKAstudios f @ t d in

LAIKA

We have to do this because RenderMan texture files go from 0 to 1, *top* to bottom, whereas the texture mapping UV's go from 0 to 1, *bottom* to top, so the texture mapping space must be “flipped” in the 't' direction before it's used to read a RenderMan texture. This must be done individually each UV Index region.

Code Examples

3. Texture Access : Texture Info

```
// Check the existence of the texture file.
int exists = 0;
gettextureinfo( File, "exists", exists );

// Number of channels.
int channels = 0;
gettextureinfo( File, "channels", channels );

if( exists )
{
    RGB = texture( File, xy.x, xy.y, "alpha", A );
    // The file is assumed to contain 1(R), 2(RA),
    // 3(RGB), or 4(RGBA) channels of information.
    if( channels < 3 )
    {
        RGB = color( RGB[0] );
        // Two channels: alpha is the second.
        if( 1 == channels )
        {
            A = RGB[1];
        }
    }
}
```

© LAIKAstudios f @ t d in

LAIKA

Next we check whether the file actually exists or not, and get the number of channels it contains, using the `gettextureinfo()` call.

The number of channels tells us what the texture file contains, and how to set the outputs based on that.

Note the `texture()` function uses the File string as-is, with the filename tokens in it, so the texture system can replace those with the correct values based on the integer portion of the xy space used to map the texture onto the surface patch.

Code Examples

3. Texture Access

```
// Set the remaining outputs.
R = RGB.r;
G = RGB.g;
B = RGB.b;
}
```



© LAIKAstudios f @ t d in

LAIKA

Then we set the float component outputs, and we're done.

Code Examples

1. Parameter Conversion
2. Spaces and Transformation
3. Texture Access
4. Procedural Patterns
5. Displacement and Bumping

© LAIKAstudios f @ t d in

LAIKA

AUTO

Code Examples

4. Procedural Patterns

- Spatial Conversion
- Noise
- Fractal
- Cellular
- Warping
- Ray Tracing

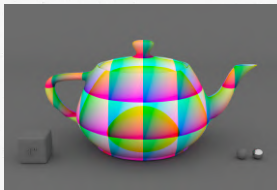
@LAIKAstudios f @ t d in

LAIKA

Now we'll cover procedural pattern generation.

Code Examples

4. Procedural Patterns : Spatial Conversion



@LAIKAstudios f @ t d in

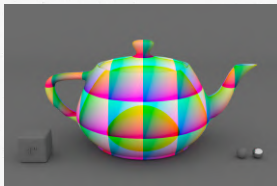
LAIKA

We've seen spatial conversion patterns in many of the previous slides. This type of pattern generator takes a continuous and unbounded value, such as a surface point, and converts it directly into a 0 to 1 ranged value.

Code Examples

4. Procedural Patterns : Spatial Conversion

```
shader pattern_sawtooth
{
    point Space = point(0.0),
    float Size = 1.0,
    output color RGB = color(0.0)
}
// Pattern generation space.
point Q = Space;
if (!connected(Space))
{
    Q = P;
    getattributed("primvar", "_Pref", Q);
    Q = transform("common", "Object", Q);
}
// Size scaling.
Q *= max(0.00001, Size);
// Sawtooth: Q-floor(Q) = mod(Q,1).
Q = Q-floor(Q);
RGB = color(Q);
}
```



@LAIKAstudios f @ t d in

LAIKA

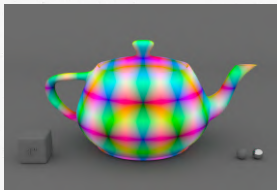
Here's the sawtooth pattern generator. We've seen this pattern used before, but here is a fully functional shader that implements it. Space is the unbounded value. Size adjusts the generated pattern's size; and RGB is the output color. It starts by determining whether to use the Space parameter's value, if it's connected to another shading node. If not it will use the Pref primvar, if it exists, or the displaced surface point P if it does not. Either way, the result of course, is transformed into "object" space before proceeding.

Then Q is scaled by the Size parameter to make the pattern bigger or smaller: when Size is larger, so is the resulting pattern, and vice versa. Finally, Q is converted into a bounded value, producing a sawtooth ramped pattern, which is output directly as a color.

Code Examples

4. Procedural Patterns : Spatial Conversion

```
shader pattern_Triangle
{
    point Space = point(0.0),
    float Size = 1.0,
    output color RGB = color(0.0)
}
...
// Triangle.
Q = 2 * min( mod(Q,1), mod(-Q,1) );
RGB = color( Q );
}
```



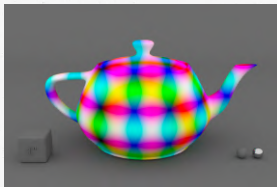
Here's a triangle pattern generator.

The mod() function is used to create two opposing sawtooth patterns. Taking the min() of them both results in a triangle-ramped pattern that ranges from 0 to .5, that's then doubled to produce a 0 to 1 ranged value.

Code Examples

4. Procedural Patterns : Spatial Conversion

```
shader pattern_Sine
{
    point Space = point(0.0),
    float Size = 1.0,
    output color RGB = color(0.0)
}
...
// Sine.
Q = linearstep( -1.0, 1.0, sin( Q*PI ) );
RGB = color( Q );
}
```



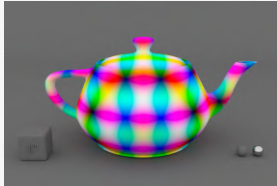
A sine wave is converted from -1 to +1 into the 0 to 1 range by the linearstep() function.

Note that sin() and linearstep() both work on tuple variable types. In this case, a point.

Code Examples

4. Procedural Patterns : Spatial Conversion

```
shader pattern_Cosine
{
    point Space = point(0.0),
    float Size = 1.0,
    output color RGB = color(0.0)
}
...
// Cosine.
Q = linearstep(-1.0, 1.0, cos(Q*PI));
RGB = color(Q);
}
```



@LAIKAstudios f @ t d in

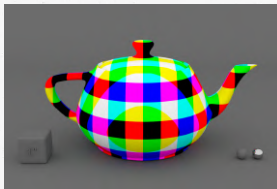
LAIKA

Cosine is similar.

Code Examples

4. Procedural Patterns : Spatial Conversion

```
shader pattern_Square
{
    point Space = point(0.0),
    float Size = 1.0,
    float DutyCycle = 0.5,
    output color RGB = color(0.0)
}
...
// Square.
Q = step(1.0-DutyCycle, Q-floor(Q));
RGB = color(Q);
}
```



@LAIKAstudios f @ t d in

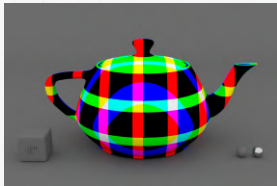
LAIKA

A square wave is generated by a step() function applied to the sawtooth pattern. This also adds a DutyCycle parameter...

Code Examples

4. Procedural Patterns : Spatial Conversion

```
shader pattern_Square
{
    point Space = point(0.0),
    float Size = 1.0,
    float DutyCycle = 0.3,
    output color RGB = color(0.0)
}
...
// Square.
Q = step(1.0-DutyCycle, Q-floor(Q));
RGB = color(Q);
}
```



@LAIKAstudios f @ t d in

LAIKA

...to make the duty cycle of the result lower...

Code Examples

4. Procedural Patterns : Spatial Conversion

```
shader pattern_Square
{
    point Space = point(0.0),
    float Size = 1.0,
    float DutyCycle = 0.7,
    output color RGB = color(0.0)
}
...
// Square.
Q = step(1.0-DutyCycle, Q-floor(Q));
RGB = color( Q );
}
```



@LAIKAstudios f @ t d a in

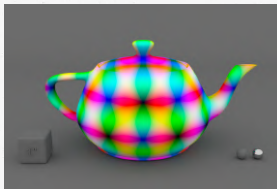
LAIKA

...or higher.

Code Examples

4. Procedural Patterns : Spatial Conversion

```
shader pattern_Smoothstep
{
    point Space = point(0.0),
    float Size = 1.0,
    float DutyCycle = 0.5,
    output color RGB = color(0.0)
}
...
// Triangle.
Q = 2 * min( mod(Q,1), mod(-Q,1) );
// End points.
float a = linearstep 0.5, 1.0, 1.0-DutyCycle;
float b = linearstep 0.0, 0.5, 1.0-DutyCycle;
// Smooth it out.
Q = smoothstep a, b, Q;
RGB = color( Q );
}
```



@LAIKAstudios f @ t d a in

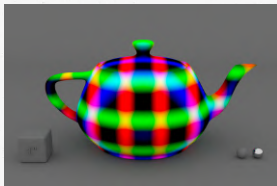
LAIKA

Using a smoothstep() function, we have the ability to control the pattern's Duty Cycle, but also get a smoothly varying pattern. It starts with a triangle waveform, the slopes of which are then smoothed out with the smoothstep() function.

Code Examples

4. Procedural Patterns : Spatial Conversion

```
shader pattern_Smoothstep
{
    point Space = point(0.0),
    float Size = 1.0,
    float DutyCycle = 0.5,
    output color RGB = color(0.0)
}
...
// Triangle.
Q = 2 * min( mod(Q,1), mod(-Q,1) );
// End points.
float a = linearstep 0.5, 1.0, 1.0-DutyCycle;
float b = linearstep 0.0, 0.5, 1.0-DutyCycle;
// Smooth it out.
Q = smoothstep a, b, Q;
RGB = color( Q );
}
```



@LAIKAstudios f @ t d a in

LAIKA

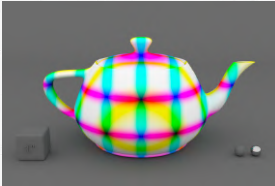
The smoothstep()'s a and b endpoints are used to control the DutyCycle of the result.

AUTO

Code Examples

4. Procedural Patterns : Spatial Conversion

```
shader pattern_Smoothstep
{
    point Space = point(0.0),
    float Size = 1.0,
    float DutyCycle = 0.7,
    output color RGB = color(0.0)
}
...
// Triangle:
Q = 2 * sin( mod(Q,1), mod(-Q,1) );
// End points:
float a = linearstep 0.5, 1.0, 1.0-DutyCycle );
float b = linearstep 0.0, 0.5, 1.0-DutyCycle );
// Smooth it out:
Q = smoothstep( a, b, Q );
RGB = color( Q );
}
```



LAIKA

@LAIKAstudios f @ t d a in

Code Examples

4. Procedural Patterns : Spatial Conversion

```
shader pattern_Smoothstep
{
    point Space = point(0.0),
    float Size = 1.0,
    float DutyCycle = 0.5,
    output color RGB = color(0.0),
    output color Gray = color(0.0),
    output float Float = 0.0
}
...
RGB = color( Q );
Float = Q.x;
Float = mix( Float, 1.0-Float, Q.y );
Float = mix( Float, 1.0-Float, Q.z );
Gray = color( Float );
}
```



LAIKA

@LAIKAstudios f @ t d a in

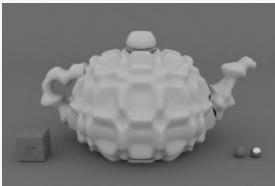
Converting the red green and blue values into a float produces some useful patterns as well.

To do this, the first dimension is inverted based on the second, and that result is then inverted based on the third.

Code Examples

4. Procedural Patterns : Spatial Conversion

```
shader pattern_Smoothstep
{
    point Space = point(0.0),
    float Size = 1.0,
    float DutyCycle = 0.3,
    output color RGB = color(0.0),
    output color Gray = color(0.0),
    output float Float = 0.0
}
...
RGB = color( Q );
Float = Q.x;
Float = mix( Float, 1.0-Float, Q.y );
Float = mix( Float, 1.0-Float, Q.z );
Gray = color( Float );
}
```



LAIKA

@LAIKAstudios f @ t d a in

Here's the float output with a DutyCycle of .3 used as a displacement.

Code Examples

4. Procedural Patterns

Spatial Conversion

Noise

Fractal

Cellular

Warping

Ray Tracing

@LAIKAstudios f @ t d a in

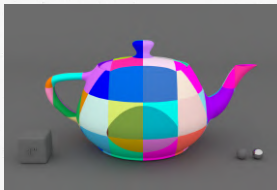
LAIKA

Now we'll look at the noise() function.

Code Examples

4. Procedural Patterns : Noise

```
shader pattern_Noise
{
    point Space = point(0.0),
    float Size = 1.0,
    string Type = "cell",
    float Variation = 0.0,
    output color RGB = color(0.0)
}
// Pattern generation space.
point Q = Space;
...
RGB = noise( Type, Q, Variation );
}
```



@LAIKAstudios f @ t d a in

LAIKA

This is noise of type “cell”, which isn’t really noise, but a uniform random value.

It produces a constant result based on the integer portion of the arguments supplied to it.

It’s typically used to generate a constant value based on an object’s Id in order to ensure that no two objects have the exact same appearance. But it’s also used in Cellular pattern generation, as we’ll see later.

Code Examples

4. Procedural Patterns : Noise

```
shader pattern_Noise
{
    point Space = point(0.0),
    float Size = 1.0,
    string Type = "uperlin",
    float Variation = 0.0,
    output color RGB = color(0.0)
}
// Pattern generation space.
point Q = Space;
...
RGB = noise( Type, Q, Variation );
}
```



@LAIKAstudios f @ t d a in

LAIKA

Perlin noise is named after Ken Perlin, its inventor. “uperlin” means the result of the noise() function is “unsigned”: that is, it ranges between 0 and 1 rather than -1 to 1.

Code Examples

4. Procedural Patterns : Noise

```
shader pattern_Noise
{
    point Space = point(0,0),
    float Size = 1.0,
    string Type = "perlin",
    float Variation = 1.0,
}
output color RGB = color(0,0)

// Pattern generation space.
point Q = Space;

...

RGB = noise( Type, Q, Variation );
}
```



@LAIKAstudios f @ t d in

LAIKA

Variation provides a fourth dimension to the pattern.

You can use it to animate the result, or to ensure the pattern is different on different objects, or simply to change the pattern.

Code Examples

4. Procedural Patterns : Noise

```
shader pattern_Noise
{
    point Space = point(0,0),
    float Size = 1.0,
    string Type = "simplex",
    float Variation = 0.0,
}
output color RGB = color(0,0)

// Pattern generation space.
point Q = Space;

...

RGB = noise( Type, Q, Variation );
}
```



@LAIKAstudios f @ t d in

LAIKA

Simplex noise was also created by Ken Perlin.

It is simpler to compute, visually isotropic, and has higher contrast than Perlin noise, which tends to only range between .2 and .8 instead of 0 and 1.

Code Examples

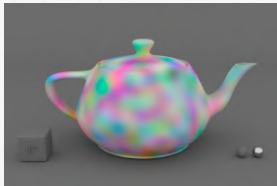
4. Procedural Patterns : Noise

```
shader pattern_Gabor
{
    point Space = point(0,0),
    float Size = 1.0,
}
output color RGB = color(0,0)

// Pattern generation space.
point Q = Space;

...

color n = noise( "gabor", Q );
RGB = 0.5*n + color(0.5);
}
```



@LAIKAstudios f @ t d in

LAIKA

Gabor noise is named after Dennis Gabor who, incidentally, won the Nobel prize for inventing holograms.

Its result is based on a filter that's applied to an underlying pattern.

Note that it does not accept a Variation argument, and that its output range is from -1 to 1, as there is no "unsigned gabor" noise type.

The implementation of Gabor noise() also does not make use the additional arguments shown for the noise() function in the OSL Specification.

AUTO

Code Examples

4. Procedural Patterns

- Spatial Conversion
- Noise
- Fractal
- Cellular
- Warping
- Ray Tracing

LAIKAstudios f t d i n

LAIKA

Code Examples

4. Procedural Patterns : Fractal

```
shader pattern_FractalNoise
{
    point Space = point(0.0),
    float Size = 1.0,
    string Type = "Superlin",
    float Variation = 0.0,
    float Dimension = 1.0,
    float Lacunarity = 2.0,
    float Erosion = 0.0,
    int Iterations = 5,
    output color RGB = color(0.0)
}
```



LAIKAstudios f t d i n

LAIKA

We'll now create a Fractal using the noise() function as its fundamental pattern.

The term “fractal” was coined by Benoit Mandelbrot to describe a class of self-similar functions, meaning they have a similar appearance at any scale at which they're viewed.

This type of pattern often appears in nature, so they're very useful.

The classic fractal parameters are Dimension and Lacunarity.

To these I've added Erosion, which is used to smooth either the low areas or high areas of the result;

and Iterations, which controls the number of times the base pattern is combined with itself.

Code Examples

4. Procedural Patterns : Fractal

```
// Pattern generation space.
point Q = Space;

// Compute the fractal.
for (int i=0; i < iterations; i++)
{
    // This iteration's frequency.
    point Qi = Q*freq;

    // Accumulate this iteration's noise pattern.
    RGB = noise(Type, Qi, Variation) * mag;

    // Update for the next iteration.
    freq *= Lacunarity;
    sum += mag;

    // Erosion adjustment.
    float avg = ( RGB.r + RGB.g + RGB.b ) * 0.3333333;
    float erosion = mix(-Erosion, Erosion, avg/sum);

    // Fractal magnitude of the next iteration.
    mag = 1.0 / pow(freq, 2.0-2.0*dimension + erosion);

    // Normalize the result.
    RGB /= sum;
}
```

LAIKAstudios f @ t d a in

LAIKA

The shader begins with the same pattern generation space used in the previous examples.

Then we initialize the fractal generation variables, beginning with the frequency of the initial pattern, which is set to 1.

Then a for() loop is used to accumulate patterns of different frequencies and magnitudes into the result.

The first two lines produce the current iteration's noise value.

Then the frequency and sum are updated for the next iteration. As you can see, Lacunarity scales the frequency of each successive iteration.

Erosion applies an offset to the magnitude by examining the currently accumulated result.

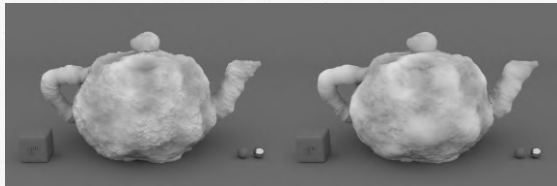
High-valued areas will have the Erosion parameter added to their frequency exponent, and low-valued areas will have the Erosion value subtracted from it.

Finally, the magnitude of the next iteration is computed using the classic Fractal equation.

Once all the iterations have been accumulated, the result is normalized so it ranges between 0 and 1.

Code Examples

4. Procedural Patterns : Fractal



LAIKAstudios f @ t d a in

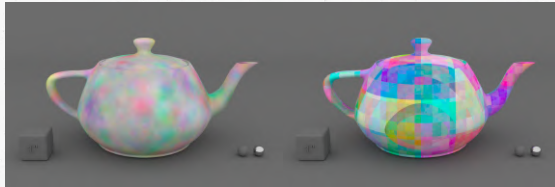
LAIKA

Using the red channel as the displacement height, you can see how the Erosion control works.

When Erosion is negative, the lower-valued areas become smoother, and when it's positive the higher-valued areas are smoother.

Code Examples

4. Procedural Patterns : Fractal



@LAIKAstudios f @ t d in

LAIKA

These examples use Simplex noise on the left, and “cell” noise on the right.

Code Examples

4. Procedural Patterns : Fractal



@LAIKAstudios f @ t d in

LAIKA

And here, a Voronoi pattern and a texture map are used. This illustrates the fact that it's possible to use any fundamental pattern and create a fractal from it, not just noise.

Code Examples

4. Procedural Patterns

- Spatial Conversion
- Noise
- Fractal
- Cellular
- Warping
- Ray Tracing

@LAIKAstudios f @ t d in

LAIKA

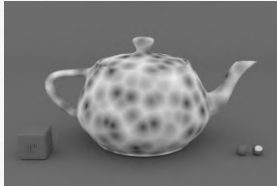
AUTO

Code Examples

4. Procedural Patterns : Cellular

```
shader pattern_Worley
{
    point Space = point(0,0),
    float Size = 1.0,
    float Jitter = 1.0,
    int Invert = 0,

    output float F1 = 0.0,
    output float F2 = 0.0,
    output float F2minusF1 = 0.0,
    output color Voronoi = color(0,0),
    output point CellPoint = point(0,0)
}
```



© LAIKAstudios f @ t d a in

LAIKA

Steven Worley created the Worley “noise”, as it’s called, which is a cellular pattern generation system because it employs a regular grid of cells as the basis for its operation.

Each “cell” is a square, for a two dimensional pattern, or a cube for a three dimensional pattern.

Within each cell, a single point is randomly placed, and it’s those randomly placed points that are used to create the resulting pattern.

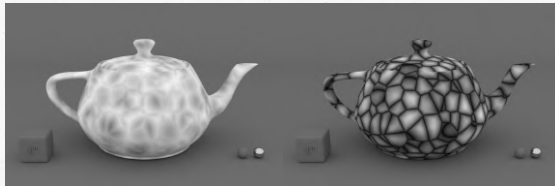
The shader uses the same Space and Size parameters as previous examples.

The Jitter parameter controls how randomly the cells’ points are placed, and an Invert switch is used to invert the results.

The F1 pattern seen here represents the distance to the nearest point within a cell.

Code Examples

4. Procedural Patterns : Cellular



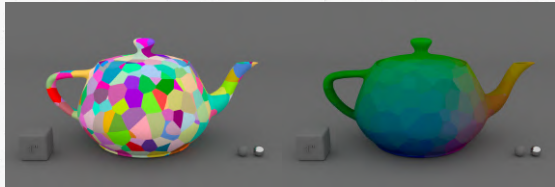
© LAIKAstudios f @ t d a in

LAIKA

F2 on the left, is the distance to the second closest cell point. And on the right is F2minusF1.

Code Examples

4. Procedural Patterns : Cellular



@LAIKAstudios f @ t d a in

LAIKA

The Voronoi output is a constant-colored pattern based on the location of the closest cell point.

And the CellPoint output is simply the location of the closest cell's jittered point.

Code Examples

4. Procedural Patterns : Cellular

```
// Pattern generation space.
point Q = Space;

if( !isconnected( Space ) )
{
    Q = P2;
    getAttribute( "primitives", "_Pret", Q );
    Q = transform( "common", "Object", Q );
}

// Size scaling.
Q *= mod( 1.0e6, Size );
```

@LAIKAstudios f @ t d a in

LAIKA

Just as in previous examples, Q is the point we're currently shading.

Code Examples

4. Procedural Patterns : Cellular

```
// Divide Q into cells (unit cubes) and randomly place
// a point within each. Then measure the distance between
// the shaded point and the random point in each of the
// size cells that contain it or are neighbors of it.
F1 = F2 = 5.0; // Ensure these will be changed.

// The shaded point's cell index.
point QIndex = floor( Q );
for( int i = -1; i <= 1; i++ )
{
    for( int j = -1; j <= 1; j++ )
    {
        for( int k = -1; k <= 1; k++ )
        {
            // The cell's index and its random offset.
            point cellIndex = QIndex + vector( i, j, k );
            vector cellOffset = mix( "cell", cellIndex );

            // The cell's jittered point.
            point cellQ = cellIndex + Jitter( cellOffset );

            // Euclidean distance measure to Q.
            float distance = length( vector( cellQ - Q ) );

            // New F1 or F2?
            // If distance < F1:
            float newF1 = stop( distance, F1 );
            // If F1 < distance < F2:
            float newF2 = mix( stop( distance, F2 ), 0.0, newF1 );

            // New F1: update both.
            F2 = mix( F2, F1, newF1 );
            F1 = mix( F1, distance, newF1 );

            // New F2: update F2.
            F2 = mix( F2, distance, newF2 );

            // Update Voronoi color.
            Voronoi = mix( Voronoi, color( cellOffset ), newF1 );

            // Update CellPoint location.
            CellPoint = mix( CellPoint, cellQ, newF1 );
        }
    }
} // End loops.
```

@LAIKAstudios f @ t d a in

LAIKA

Then F1 and F2 are initialized to a value higher than is possible to ensure they'll be changed later.

We then find out what cell Q is in.

The for() loops are used to check all the neighboring cells plus the current cell, so the distance from the current point, Q, and the jittered cell point, can be measured. In this case we're using the Euclidean distance.

In the next column, the distance value is used to update F1 and F2 as needed, along with the Voronoi and CellPoint outputs.

Code Examples

4. Procedural Patterns : Cellular

```
// Additional float output.  
F2minusF1 = F2 - F1;  
{  
  if ( Invert )  
  {  
    F1 = 1.0 - F1;  
    F2 = 1.0 - F2;  
    F2minusF1 = 1.0 - F2minusF1;  
  }  
}
```

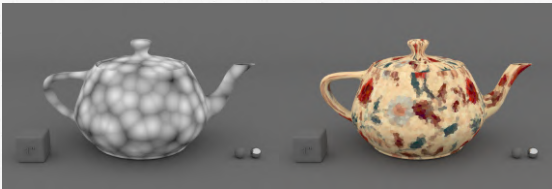
@LAIKAstudios f @ t d in

LAIKA

We finish up with the F2minusF1 output, and apply the Inversion if it was set.

Code Examples

4. Procedural Patterns : Cellular



@LAIKAstudios f @ t d in

LAIKA

The Inverted F1 output looks like this, and the CellPoint value, when it's been properly scaled and is used as the Space input to another pattern, in this case a texture, can be used to create a mosaic effect.

Code Examples

4. Procedural Patterns

- Spatial Conversion
- Noise
- Fractal
- Cellular
- Warping
- Ray Tracing

@LAIKAstudios f @ t d in

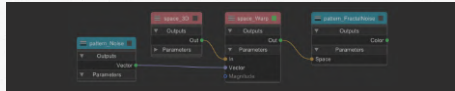
LAIKA

AUTO

Code Examples

4. Procedural Patterns : Warping

```
shader space_Warp
{
    point In = point(0,0);
    vector Vector = vector(0,0);
    int ExpandRange = 0;
    float Magnitude = 0.5;
    output point Out = point(0,0)
}
{
    vector dir = select( Vector, 2*Vector-Vector(), ExpandRange );
    Out = In + dir*Magnitude;
}
```



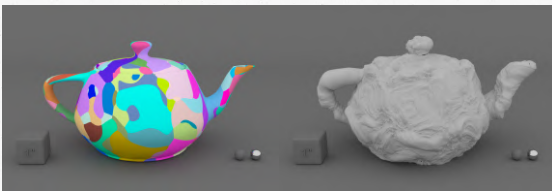
@LAIKAstudios f @ t d a in

LAIKA

Warping is a simple but effective tool for adding distortion to a pattern. It simply takes an Input point and adds the Vector parameter value to it, making sure the Vector's range is appropriate by expanding it if necessary, and then scales it by the Magnitude. When using this shader, a Vector-producing node is used to Warp the Space being used to generate the desired pattern.

Code Examples

4. Procedural Patterns : Warping



@LAIKAstudios f @ t d a in

LAIKA

The result depends entirely on the relative variations and sizes of the warping Vector, the Pattern being warped, and the Warp's Magnitude.

Code Examples

4. Procedural Patterns

- Spatial Conversion
- Noise
- Fractal
- Cellular
- Warping
- Ray Tracing

@LAIKAstudios f @ t d a in

LAIKA

AUTO

Code Examples

4. Procedural Patterns : Ray Tracing

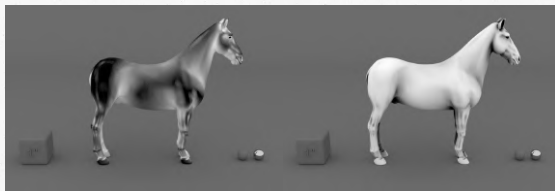
@LAIKAstudios f @ t d a in

LAIKA

We're now going from Warping, one of the simplest pattern generation techniques, to Ray Tracing, which is one of the most complicated - at least from a programming standpoint.

Code Examples

4. Procedural Patterns : Ray Tracing



@LAIKAstudios f @ t d a in

LAIKA

The example I'm going to show you is Occlusion. Occlusion is based on tracing rays into the hemisphere above the surface, and counting what percentage of those rays hit something.

Looking at the left horse, you can see that in areas where the surface is more closely surrounded by the rest of the object, the occlusion is brighter, since a greater percentage of rays will hit another surface. On the right, the pattern is simply inverted.

You can use Occlusion to determine areas where dirt might accumulate, or its inverse to control where the surface might experience more wear.

Code Examples

4. Procedural Patterns : Ray Tracing

```
shader pattern_Occlusion
{
    vector TraceDirection = vector(0,0),
    float TraceBias = 0.0,
    string TraceSet = "",
    int Samples = 10,
    float Radius = 1.0,
    float Contrast = 0.5,
    output float Occlusion = 0.0,
    output float Dirt = 0.0,
    output float Wear = 0.0,
    output vector NormalRay = vector(0,0)
}
```



@LAIKAstudios f @ t d a in

LAIKA

The shader interface starts with the Trace Direction, Bias, and object Set parameters.

TraceDirection is the central axis used to generate the hemisphere of random ray directions, and is typically the surface normal.

TraceBias is a small offset from the ray origin that's used to ensure the ray doesn't accidentally intersect the surface it originates from.

And TraceSet controls the set of objects the traced rays can intersect. If it's blank, all objects in the scene can be hit - not just the object being shaded.

Then come parameters that control the results. Samples determines the number of traced rays that will be used for each shaded point. Higher

numbers improve the quality, but of course take longer to process. MaxDist sets the farthest distance a ray can travel - if it doesn't hit anything within that range, the ray is considered a miss. And Contrast adjusts the contrast of the Dirt and Wear outputs. The Occlusion output is simply the computed occlusion, Dirt and Wear are the Contrast-adjusted occlusion and inverse occlusion, and BentNormal is the average direction of all the rays that *didn't* hit anything. It represents a direction pointing toward the “open sky”, or un-occluded view above the surface.

Code Examples

4. Procedural Patterns : Ray Tracing

```
// Trace direction.
vector Dn = N;

if (isconnected(TraceDirection) || vector(0,0) != TraceDirection)
{
    Dn = normalize(TraceDirection);
}

// Deff, T. et. al. 2007
// Building an Orthonormal Basis, Revisited
// Journal of Computer Graphics Techniques Vol. 6, No. 1
// https://jcg.org/publications/0006/01/01/
vector Tn, Bn;

float signz = select(-1, 1, Dn.z >= 0.0);
float s = -1.0 / (1.0 + Dn.z);
Bn = vector(a*Dn.x*Dn.y, signz + a*Dn.y*Dn.y, -Dn.y);
Tn = vector(1.0 + a*signz*Dn.x*Dn.x, signz*Dn.x, -signz*Dn.x);
```

LAIKAstudios f @ t d in

LAIKA

First we determine the trace direction, which is N if the parameter hasn't been connected or explicitly set to anything. Then the orthonormal Tangent and Bi-tangent vectors are created from it. These three vectors are all perpendicular to each other, and are essential to creating the random ray directions that will be traced.

Code Examples

4. Procedural Patterns : Ray Tracing

```
// Trace bias.
float bias = 0.0001;

if (isconnected(TraceBias) || 0.0 != TraceBias)
{
    bias = TraceBias;
}
else
{
    int autobias = 0;
    getattribute("trace:autobias", autobias);
    if (autobias) getattribute("builtin", "bias", bias);
    else getattribute("trace:bias", bias);
}
```

```
// Trace set.
string traceSet = TraceSet;

if ("*" == TraceSet)
{
    getattribute("identifier:name", traceSet);
}
else if ("-" == TraceSet)
{
    getattribute("identifier:name", traceSet);
    traceSet = concat("-", traceSet);
}
```

LAIKAstudios f @ t d in

LAIKA

Trace Bias is set by the parameter, or is automatically determined by the renderer; using either the builtin bias, or the “trace:bias” attribute. The TraceSet is determined by its parameter value, which is a comma-separated list of object group identifiers that specifies what object groups are to be used. Note that we can also use an asterisk to use the object's identifier:name attribute as the trace group, in which case only the object being shaded will participate in the occlusion computation, or minus-asterisk to use all objects except the object being shaded.

Code Examples

4. Procedural Patterns : Ray Tracing

```
// Trace samples and accumulate the results.
float hits = 0;
float hitdists = 0;
vector missdirs = vector(0);

for( int i=0; i < Samples; i++ )
{
    // Generate a uniform sample direction
    // in the positive Dn hemisphere only.
    vector dir;
    {
        point xi = hashnoise( P, float(i) ); // ξ : [0,1]
        float phi = xi.x * 2*M_PI; // θ = φ < 2π
        float sinphi, cosphi;
        sincosf( phi, sinphi, cosphi );
        float z = xi.y;
        float sintheta = sqrt( 1.0 - z*z ); // θ = θ < π
        float x = sintheta * cosphi;
        float y = sintheta * sinphi;
        dir = x*Dn + y*Dn + z*Dn;
    }

    // Trace the sample ray.
    int hit = trace( P, dir,
                    "handist", hndist,
                    "handist", hndist,
                    "tracedist", tracedist );

    // Accumulate the results.
    hits += hit;
    missdirs += (1-hit) * dir;

    float hitdist;
    getmessage( "trace", "hitdist", hitdist );
    hitdists += hitdist;
}

// End Samples loop.
```

LAIKAstudios f @ t d a in

LAIKA

Now we generate and accumulate the ray-traced samples.

The loop starts by generating a uniform random direction. The hashnoise() function produces a new set of uniform random values each time through the loop.

The rest of the code block uses two of those random numbers to produce two spherical coordinate angles that define the vector's direction.

In the next column we call the trace() function to trace the sample ray, using the optional arguments to control it.

Once the ray has been traced, the results are tallied.

Note that the trace() function itself doesn't return the traced distance, but only whether the ray hit something or not.

To get the distance to the hit surface, getmessage() is used.

Code Examples

4. Procedural Patterns : Ray Tracing

```
// Set the outputs.
Occlusion = hits / Samples;
Dirt = linearstof( 0.0, 1.0-Contrast, Occlusion );
Wear = 1.0 - Dirt;
BentNormal = normalize( missdirs );
```

LAIKAstudios f @ t d a in

LAIKA

At this point, we've gathered all the information we need to generate the outputs.

As you can see, Contrast adjusts the upper-limit of a ramp that's applied to the raw Occlusion value.

And Wear is just the inverse of Dirt.

Code Examples

1. Parameter Conversion
2. Spaces and Transformation
3. Texture Access
4. Procedural Patterns
5. Displacement and Bumping

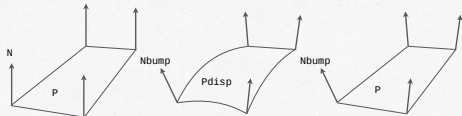
Code Examples

5. Displacement and Bumping
 - Height and Vector Patterns
 - Surface Gradient and Shading Normal

The last section covers surface displacement and bumped shading normals.

Code Examples

5. Displacement and Bumping : Height and Vector Patterns



Recall that a bumped shading normal is produced by a displaced surface point, even if the rendered surface itself is not actually displaced. So displacement and bumping both start with a displaced surface point.

Code Examples

5. Displacement and Bumping : Height Pattern

```
shader displace_height
{
    float Height = 0.5;
    float Offset = 0.0;
    float Magnitude = 1.0;
    string MagnitudeSpace = "world";
    int Invert = 0;
    output vector DeltaP = vector(0.0)
}
```

Beginning with the Height-based shader, we have the Height parameter, which defines how high the original surface point will be displaced along the surface normal.

Offset is added to the Height, and gives us an additional control to shift the resulting displacement up or down as needed.

Magnitude is a scale applied to the Height. Since the Height pattern's values are assumed to range between 0 and 1, the Magnitude is used to control how much displacement is actually produced. MagnitudeSpace defines in what coordinate system the Magnitude is measured.

Invert simply flips the displacement, making the peaks into valleys and vice versa.

And the result, rather than being a new displaced surface point, is instead the change in the surface point's location: its Delta vector.

Code Examples

5. Displacement and Bumping : Height Pattern

```
{
    // Invert?
    float height = Height;
    if (Invert) height = 1.0-height;
    // [0,1] Height value shifts the surface at 0.5, plus the Offset.
    height = height-0.5 + Offset;
    // Apply the Magnitude.
    height *= Magnitude;
}
```

The shader starts by Inverting the Height if necessary.

Then the Height value is offset by one-half so that, by itself, the Height variations will produce peaks and valleys above and below the original surface.

Offset is then simply added to that to shift the result up or down by a fixed amount.

Then Magnitude scales the result so the height value represents the variation in Magnitude Space.

Code Examples

5. Displacement and Bumping : Height Pattern

```
// Scale height to common space.
matrix mag_to_common;

if( getmatrix( MagnitudeSpace, "common", mag_to_common ))
{
    height *= pow( abs( determinant( mag_to_common ) ), 0.33333333 );
}
else
{
    warning(
        "Magnitude Space = \"%s\" couldn't be found. -\n",
        "Using \"common\" space instead - that's not good. ",
        MagnitudeSpace
    );
}

// The change in the surface point P.
DeltaP = N * height;
```

@LAIKAstudios f @ t d in

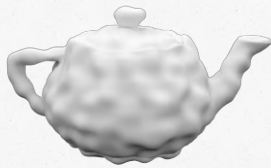
LAIKA

The height variable now must be converted to “common” space, since the surface normal is specified in “common” space, and we’re going to scale N by height to define the change in the surface point.

Describing the displacement as the change in P is extremely beneficial because it makes combining displacements trivial, as any number of them can simply be added or blended together in any way you want, and the result will accurately describe the combined set of displacements.

Code Examples

5. Displacement and Bumping : Height Pattern



@LAIKAstudios f @ t d in

LAIKA

Height displacement results in surfaces that look like this, with the displaced surface raised and lowered to form peaks and valleys.

Code Examples

5. Displacement and Bumping : Vector Pattern

```
shader displace_vector
{
    vector Vector = vector(0.0);
    float Magnitude = 1.0;
    string MagnitudeSpace = "world";
    int Repeat = 0;

    output vector DeltaP = vector(0.0)
}
```

@LAIKAstudios f @ t d in

LAIKA

Vector displacement replaces the Height and Offset parameters with a Vector input. All other parameters remain the same.

Code Examples

5. Displacement and Bumping : Vector Pattern

```
{  
  // Invert?  
  vector deltaP = Vector;  
  if (Negate) deltaP = -deltaP;  
  // Apply the Magnitude.  
  deltaP *= Magnitude;  
}
```

@LAIKAstudios f @ t d in

LAIKA

The Vector's values are assumed to be omnidirectional, so Invert simply negates the Vector.

Code Examples

5. Displacement and Bumping : Vector Pattern

```
// Scale to common space.  
matrix mag_to_common;  
if (getmatrix( MagnitudeSpace, "common", mag_to_common ))  
{  
  deltaP *= pow( abs( determinant( mag_to_common ) ), 0.33333333 );  
}  
else  
{  
  warning(  
    "Magnitude Space = \"No\" couldn't be found. -"  
    "Using \"common\" space instead - that's not good. -",  
    MagnitudeSpace  
  );  
}  
// The change in the surface.  
deltaP = deltaP;
```

@LAIKAstudios f @ t d in

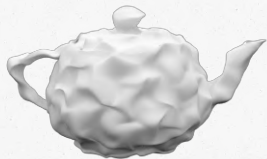
LAIKA

And once we've scaled it from MagnitudeSpace to "common" space, we're done.

Note that the Vector displacement *is* the DeltaP value. In other words, the DeltaP output *is* a Vector Displacement.

Code Examples

5. Displacement and Bumping : Vector Pattern



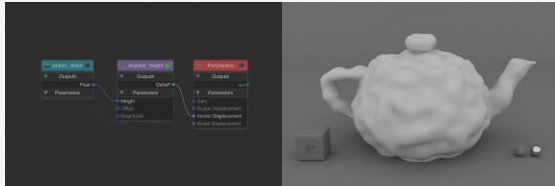
@LAIKAstudios f @ t d in

LAIKA

Vector displacement results in surfaces that look like this, where the displaced surface has been pushed and pulled in any direction.

Code Examples

5. Displacement and Bumping



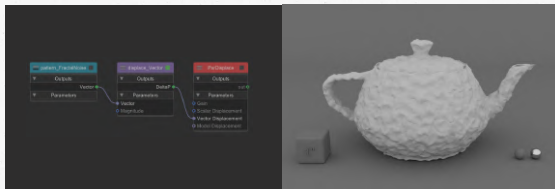
@LAIKAstudios f @ t d in

LAIKA

Since DeltaP is a vector that represents the displacement, it's connected directly to the Vector Displacement input of PxrDisplace. That's true whether DeltaP originated from a displacement Height...

Code Examples

5. Displacement and Bumping



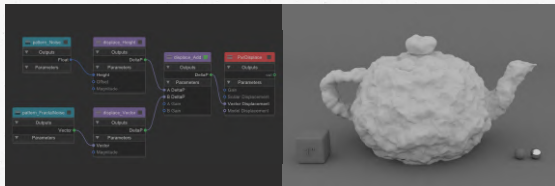
@LAIKAstudios f @ t d in

LAIKA

...or a Vector displacement...

Code Examples

5. Displacement and Bumping



@LAIKAstudios f @ t d in

LAIKA

...or is a combination of DeltaP vectors. This is the beauty of representing the displaced surface as the change in the surface point rather than as the displaced surface point itself.

AUTO

Code Examples

5. Displacement and Bumping

Height and Vector Patterns

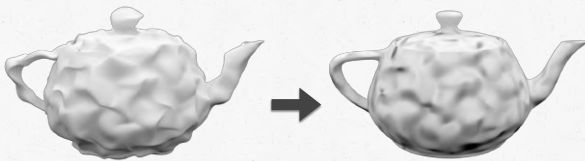
Surface Gradient and Shading Normal

@LAIKAstudios f @ t d in

LAIKA

Code Examples

5. Displacement and Bumping : Surface Gradient and Shading Normal



@LAIKAstudios f @ t d in

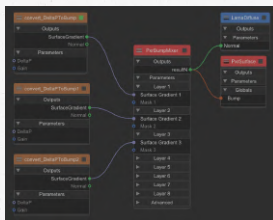
LAIKA

Let's now look at how to convert DeltaP into a bumped shading Normal rather than using it for displacement.

Code Examples

5. Displacement and Bumping : Surface Gradient and Shading Normal

```
shader convert_DeltaPtoBump
{
    vector DeltaP = vector(0.0),
    float Gain = 1.0,
    output vector SurfaceGradient = vector(0.0),
    output normal Normal = normal(0.0)
}
```



@LAIKAstudios f @ t d in

LAIKA

This conversion shader produces both a Surface Gradient vector and a bumped shading Normal.

SurfaceGradient vectors can be combined in any way you like, and the result will represent their combination; just like using a DeltaP vector to represent the displaced surface allows you to easily combine displacements.

Code Examples

5. Displacement and Bumping : Surface Gradient and Shading Normal

```
shader convert_DeltaToBump
{
    vector DeltaP = vector(0.0),
    float Gain = 1.0,
    output vector SurfaceGradient = vector(0.0),
    output normal Normal = normal(0.0)
}
```



LAIKAstudios f @ t d a in

LAIKA

The bumped shading Normal output can be connected directly to the Normal input of any Lama shading node, or the Bump input of PxrSurface.

Code Examples

5. Displacement and Bumping : Surface Gradient and Shading Normal

```
shader convert_DeltaToBump
{
    vector DeltaP = vector(0.0),
    float Gain = 1.0,
    output vector SurfaceGradient = vector(0.0),
    output normal Normal = normal(0.0)
}

// Must reference P for auto-differentiation to work.
point Po = P;
getattribute("builtin", "Po", Po);

// Compute the bumped normal from the displaced point.
point Pdisp = Po + DeltaP;
normal Nbump = calculateNormal(Pdisp);

// Mikkelsen, R. 2008
// Surface Gradient-Based Bump Mapping Framework
// Journal of Computer Graphics Techniques, Vol. 9, No. 3
// https://cit.ingrpub.com/2009/03/04/
float k = dot(N, Nbump);
SurfaceGradient = (1.0 - Nbump) / max(1.0e-6, abs(k));

Normal = normalize(N + SurfaceGradient * Gain);
}
```

LAIKAstudios f @ t d a in

LAIKA

The shader starts with the original surface point P_o and adds ΔP to it. The `calculateNormal()` function then uses that displaced surface point to generate its corresponding shading normal: which is N_{bump} . Note that N_{bump} isn't output directly as the bumped shading Normal. Instead it's first converted into a Surface Gradient representation, and then back into a shading Normal.

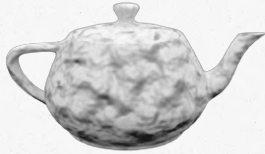
The reasons for this are thoroughly described in Mikkelsen's paper, but the primary one is that the Surface Gradient cannot result in a "back-facing" normal.

Such a Normal faces in opposition to the surface, and therefore produces a completely un-lit response in those areas where it is back-facing, which is clearly not desirable.

Code Examples

5. Displacement and Bumping : Surface Gradient and Shading Normal

```
shader convert_SG2N
{
    vector SurfaceGradient = vector(0.0),
    float Gain = 1.0,
    output normal Normal = normal(0.0)
}
Normal = normalize( N - SurfaceGradient*Gain );
```



@LAIKAstudios f @ t d in

LAIKA

If you've combined several Surface Gradients together in your shading network, it's trivial to convert that result into a bumped shading Normal.

RenderMan OSL

Code Examples

Interface Design

Development and Deployment

@LAIKAstudios f @ t d in

LAIKA

AUTO

Interface Design

1. Parameter Names and Pages
2. Widgets
3. Types and Connections
4. Help Content

@LAIKAstudios f @ t d in

LAIKA

At this point we've seen many examples of OSL shader code, but none of them had a real User Interface - they simply declared the input and output parameter variables with their default values.

In this section, I'll cover how to design a user interface that makes a shader easier to understand and control, and therefore easier to use.

Interface Design

1. Parameter Names

BaseColor	SurfaceGradient	SizeSpace
Metallic	Normal	Blend
DielectricFace	Negate	PrimitiveVariable
TraceBias	Invert	ExpandRange
MetallicAlbedo	Magnitude	DutyCycle
DeltaP	Coordinates	Space

@LAIKAstudios f @ t d a in

LAIKA

When developing a library of shaders, it's extremely helpful for you, and your users, if you employ a strict naming convention for all your shader's parameters.

There are many possibilities for how to do this, but the convention I have landed on after many years of writing these things is to use Pascal-Case. This avoids collisions with the global OSL variables or reserved words, it makes parameter variables distinct from program variables (since I use lower-case for those), and it makes them easily readable.

Whatever convention you decide to use, the main point is, once you pick one, stick to it *without exception*.

That may sound overly restrictive; but I guarantee, it will make your life, and your users', so much easier if you do.

Interface Design

1. Parameter Names

```
color SpecularEdge = color(1.0)
[
  string label = "Specular Edge (F90)"
],

float TraceBias = 0.0
[
  string label = "Trace Bias (auto)"
],

color MetallicAlbedo = color(0.0)
[
  string label = "Metallic Albedo"
],

vector DeltaP = vector(0.0)
[
  string label = "Delta P"
],

string PrimitiveVariable = "st"
[
],

vector TraceDirection = vector(0.0)
[
  string label = "Trace Direction (N)"
],

int ExpandRange = 0
[
  string label = "Expand Range"
],

float Blend = 0.5
[
  string label = "Blend"
],
```

@LAIKAstudios f @ t d a in

LAIKA

When naming parameter variables, I try to think of names that are descriptive of what the parameter does, what it contains, or what output or effect it controls.

Once you've done that, you can add metadata to the parameter declaration to make it more readable.

Metadata is contained within double square brackets.

The "label" hint contains a string that the application should display for the user instead of the variable name itself.

I use labels to insert spaces between separate words.

I also sometimes add a hint, in parenthesis, about the parameter, or its

default value, if that differs from its initial value.

Above all, avoid the urge to make the label different from the variable name itself - that is *not* a feature, and will only lead to confusion.

Interface Design

1. Parameter Names and Pages

```
float TextureAccess_Blur = 0.0
[[
  string page = "Texture Access",
  string label = "Blur"
]],

float WeavePattern_CountMultiplier = 1.0
[[
  string page = "Weave Pattern",
  string label = "Count Multiplier"
]],

int NormalComputation_InvertProfile = 0
[[
  string page = "Normal Computation",
  string label = "Invert S Profile"
]],

float WarpThread_Ply_Twist = 20.0
[[
  string page = "Warp Thread.Ply",
  string label = "Twist"
]],
```



@LAIKAstudios f @ t d in LAIKA

In addition to labeling parameters, you can organize them into hierarchies. This is done with the “page” hint, and is useful when you have a large or complex interface with many parameters, or it simply makes sense to group certain parameters under a single heading. When using pages, the convention I use is to prepend the page name onto the parameter variable’s name with an underscore. This is helpful to prevent parameter name collisions between different pages, and makes it easy for your user to know exactly what the parameter’s real name is, just by looking at the shader’s User Interface in a RenderMan Bridge application.

Interface Design

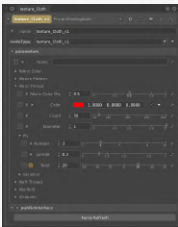
1. Parameter Names and Pages

```
float TextureAccess_Blur = 0.0
[[
  string page = "Texture Access",
  string label = "Blur"
]],

float WeavePattern_CountMultiplier = 1.0
[[
  string page = "Weave Pattern",
  string label = "Count Multiplier"
]],

int NormalComputation_InvertProfile = 0
[[
  string page = "Normal Computation",
  string label = "Invert S Profile"
]],

float WarpThread_Ply_Twist = 20.0
[[
  string page = "Warp Thread.Ply",
  string label = "Twist"
]],
```



@LAIKAstudios f @ t d in LAIKA

You can also have pages within pages if you really need to, by separating the page names with a period.

AUTO

Interface Design

1. Parameter Names and Pages
2. Widgets
3. Types and Connections
4. Help Content

@LAIKAstudios f @ t d in

LAIKA

Interface Design

2. Widgets

checkBox
mapper
popup
slider
assetIdInput

@LAIKAstudios f @ t d in

LAIKA

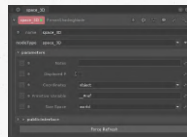
Widgets control how each shader parameter is presented.

Interface Design

2. Widgets

checkBox
mapper
popup
slider
assetIdInput

```
int DisplacedP = 0  
{  
  string label = "Displaced P",  
  string widget = "checkBox",  
}
```



@LAIKAstudios f @ t d in

LAIKA

A checkBox works exactly how you'd expect: the UI presents a box that can be checked on or off.

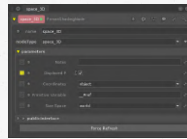
If the parameter is 0, the box is un-checked...

Interface Design

2. Widgets

```
checkBox
mapper
popup
slider
assetIdInput
```

```
int Displaced = 0
[[
    string label = "Displaced",
    string widget = "checkBox",
]],
```



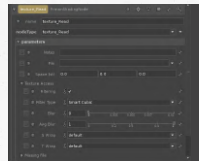
...and when checked, the parameter is set to 1.

Interface Design

2. Widgets

```
checkBox
mapper
popup
slider
assetIdInput
```

```
int TextureAccess_Strip = 0
[[
    string page = "Texture Access",
    string label = "S Strip",
    string widget = "mapper",
    string options = "default:1"
    "clamp:1"
    "periodic:2"
    "mirror:3"
]],
```



A mapper widget displays any number of option strings, and associates each with a value.

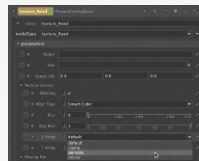
The options are separated from each other by a vertical bar.

Interface Design

2. Widgets

```
checkBox
mapper
popup
slider
assetIdInput
```

```
int TextureAccess_Strip = 0
[[
    string page = "Texture Access",
    string label = "S Strip",
    string widget = "mapper",
    string options = "default:1"
    "clamp:1"
    "periodic:2"
    "mirror:3"
]],
```



When the user selects one of the options, the parameter is set to the selected value.

Note that in OSL, literal strings are automatically concatenated together.

The assetIdInput is a RenderMan widget used for accessing texture files. Selecting Browse...

2. Widgets

```
checkboxBox
mapper
popup
slider
assetIdInput

string file = ""
{
    string widget = "assetIdInput",
    string options = "texture",
    string assetIdTypeTags = "texture",
    string fileTypes = "tex"
},
}
```

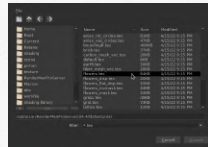


...pops open a file browser window, where you can select the texture file you want.

2. Widgets

```
checkboxBox
mapper
popup
slider
assetIdInput

string file = ""
{
    string widget = "assetIdInput",
    string options = "texture",
    string assetIdTypeTags = "texture",
    string filetypes = "tex"
},
}
```



Interface Design

1. Parameter Names and Pages
2. Widgets
3. Types and Connections
4. Help Content

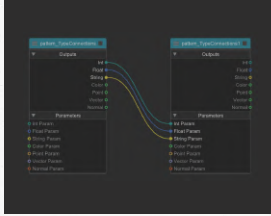
AUTO

Interface Design

3. Types and Connections

Single

int, float, string



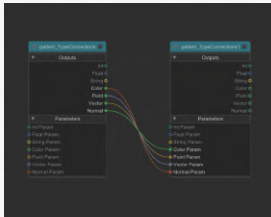
Single-valued types, such as int, float, or string, can only be connected to a parameter of the same type.

Interface Design

3. Types and Connections

Tuple

color, point,
vector, normal



However, tuple-valued types, such as color, point, vector, and normal, can be connected to any other tuple-type.

Interface Design

1. Parameter Names and Pages

2. Widgets

3. Types and Connections

4. Help Content

AUTO

Interface Design

4. Help Content

Notes Parameter
Character Styles
HTML Formatting

@LAIKAstudios f @ t d a in

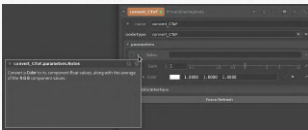
LAIKA

Help content for each of your parameters, and the shader as a whole, is extremely important to those who will be using your shaders. Without clear and detailed descriptions of what the shader and its parameters do, anyone using it will have to resort to needless experimentation to figure out for themselves how to use it.

Interface Design

4. Help Content : Notes Parameter

```
shader convert_CtoF
{
    string Notes = ""
    [
        int connectable = 0,
        string help =
            "Convert a <b>Color</b> to its component float values, "
            "along with the average of the <b>R</b> <b>G</b> <b>B</b> component values. "
    ],
}
```



@LAIKAstudios f @ t d a in

LAIKA

I start every shader with a string parameter called Notes, and I use its help content to describe what the shader does. This also provides a convenient place for the user to write their own notes to themselves or others about how they're using the shader in a larger network. Since the Notes parameter is never accessed by the shader's code, its existence remains entirely within the Bridge Application and has no effect on the shader's execution.

Interface Design

4. Help Content : Notes Parameter

```
shader convert_CtoF
{
    string Notes = ""
    [
        int connectable = 0,
        string help =
            "Convert a <b>Color</b> to its component float values, "
            "along with the average of the <b>R</b> <b>G</b> <b>B</b> component values. "
    ],
    float Gain = 1.0
    [
        int slider = 1, float slidermin = 0.1, float slidermax = 10.0, float slidercenter = 1.0,
        string help =
            "<b>Gain</b> scales the <b>Color</b> value. "
    ],
    color Color = color(1.0)
    [
        string help =
            "The <b>Color</b> value to convert. "
    ],
}
```

@LAIKAstudios f @ t d a in

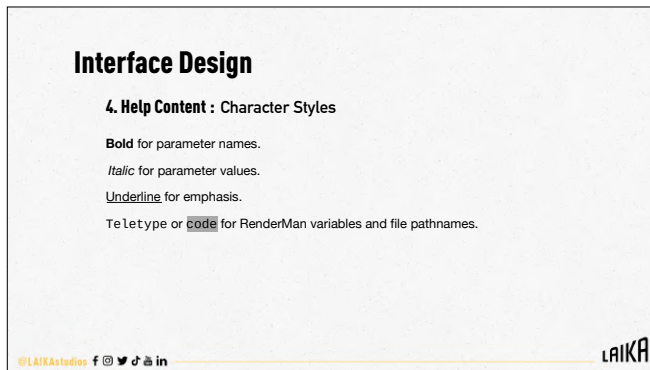
LAIKA

While the Notes parameter documents the shader, every shader parameter is also documented with its own help string.

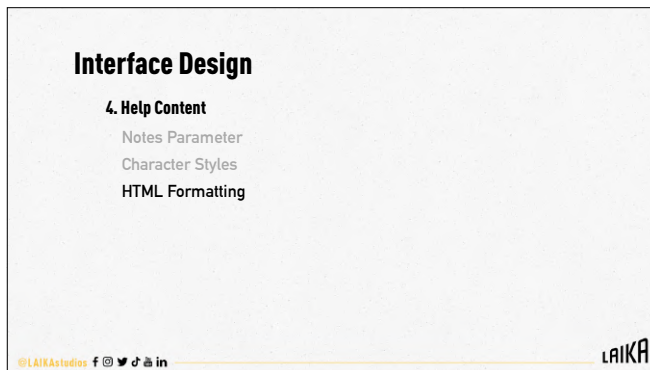
AUTO



I use different character styles to distinguish between referenced parameter names versus parameter values, or to emphasize important details. Teletype or code fonts are used to indicate external variables, such as RenderMan primvars, options, and attributes, or texture files.



AUTO



Interface Design

4. Help Content : HTML Formatting

`<b>Bold</b>` for parameter names.

`<i>Italic</i>` for parameter values.

`<u>Underline</u>` for emphasis.

`<tt>Teletype</tt>` or `<code>code</code>` for RenderMan variables and file pathnames.

```
int HelpExample = 0
{
  string help =
    "When <b>Color</b> is set to <i>(0,0,0)</i>, "
    "the primer <code>code</code> will be used. "
}
```



@LAIKAstudios f @ t d in

LAIKA

The help metadata is just a string, so formatting is done with html tags.

Interface Design

4. Help Content : HTML Formatting

`<b>Bold</b>` for parameter names.

`<i>Italic</i>` for parameter values.

`<u>Underline</u>` for emphasis.

`<tt>Teletype</tt>` or `<code>code</code>` for RenderMan variables and file pathnames.

`<font color='orange'>Note</font>`

`<br/>` `<p>``</p>`

`<a href='https://rmanwiki.pixar.com'>Link</a>` to additional documentation.

@LAIKAstudios f @ t d in

LAIKA

In addition to text styles, you can use font colors, line breaks, paragraphs, and specify links. These and other formatting capabilities are specific to the RenderMan Bridge Application you're using, so you may have to experiment.

RenderMan OSL

Code Examples

Interface Design

Development and Deployment

@LAIKAstudios f @ t d in

LAIKA

AUTO

Development and Deployment

1. Source Organization
2. Make System
3. Debugging

@LAIKAsudios f @ t d in

LAIKA

In this section I'll cover the details of creating a library of shaders, a system to build them, and some debugging techniques.

Development and Deployment

1. Source Organization



@LAIKAsudios f @ t d in

LAIKA

Shaders are defined by their source code, so a good organizational system makes them easy to find and manage, especially during the build and deployment process.

Development and Deployment

1. Source Organization

shading

@LAIKAsudios f @ t d in

LAIKA

The system I use is based on categorizing shaders by their type and functionality.
It starts with the containing directory; in this case it's called shading.

Development and Deployment

1. Source Organization

```
shading
osl
```

@LAIKAstudios f @ t d in

LAIKA

Within that are separate directories for different types of source code. At Laika, there are many types of code used for many different shading-related purposes, so each of those has its own directory. Since we're only discussing OSL shader code at the moment, that's all there is for now.

Development and Deployment

1. Source Organization

```
shading
osl
  convert
  data
  displace
  pattern
  space
  texture
```

@LAIKAstudios f @ t d in

LAIKA

Under that, the osl shaders are grouped into different directories based on their functional category.

Development and Deployment

1. Source Organization

```
shading
osl
  convert
    amp005p.osl
    Ctor.osl
    OutcropBump.osl
    PProc.osl
    PStat.osl
    Ifrog.osl
    SPETUM.osl
  data
    AttrObj.osl
    Mullin.osl
    Pruner.osl
  displace
    Add.osl
    Height.osl
    Mix.osl
    Vector.osl
  pattern
    FractalNoise.osl
    Lattice.osl
    Noise.osl
    Occlusion.osl
    Worley.osl
  space
    2d.osl
    3d.osl
    Warp.osl
    xform.osl
  texture
    Read.osl
```

@LAIKAstudios f @ t d in

LAIKA

Within each sub-directory are the OSL source files. I also use PascalCase for naming shader source files, since these filenames will become the shader names you see in the RenderMan Bridge application.

Development and Deployment

1. Source Organization
2. Make System
3. Debugging

@LAIKAstudios f @ t d in

LAIKA

To build these shaders, I use a simple Make system that will update them whenever their source file has changed.

Development and Deployment

2. Make System

```
shading
osl
  convert
  data
  displace
  pattern
  space
  texture
```

@LAIKAstudios f @ t d in

LAIKA

So in addition to the shader source sub-directories...

Development and Deployment

2. Make System

```
shading
build
  Makefile
  make
  osl
    convert
    data
    displace
    Makefile
  pattern
  space
  texture
```

@LAIKAstudios f @ t d in

LAIKA

...there is a top-level build directory where the compiled shaders are placed, a top-level Makefile to build everything, a make sub-directory that contains a script that's run during the build process, and a Makefile in the osl sub-directory.

Development and Deployment

2. Make System

```
shading
build
Makefile
make
osl
convert
data
displace
Makefile
pattern
space
texture
```

@LAIKAstudios f @ t d in

LAIKA

When the ‘make’ command is executed in the shading directory, the top-level Makefile first recursively enters all the sub-directories whose contents must be built - in this case just the osl directory - and executes their Makefiles.

Development and Deployment

2. Make System

```
shading
build
Makefile
make
osl
convert
data
displace
Makefile
pattern
space
texture
```

@LAIKAstudios f @ t d in

LAIKA

The osl/Makefile then recursively builds the contents of all the osl sub-directories...

Development and Deployment

2. Make System

```
shading
build
Makefile
make
osl
Makefile
```

```
convert
  convert.osl
  demoSRE.oso
  CtoF.oso
  DeltaProDump.oso
  FtoC.oso
  FtoI.oso
  ItoF.oso
  SREtoAK.oso
data
  Attrib.osl
  Attrib.osl
  Bullin.oso
  prjwar.oso
displace
  Add.oso
  Mix.oso
  Height.oso
  Vector.oso
```

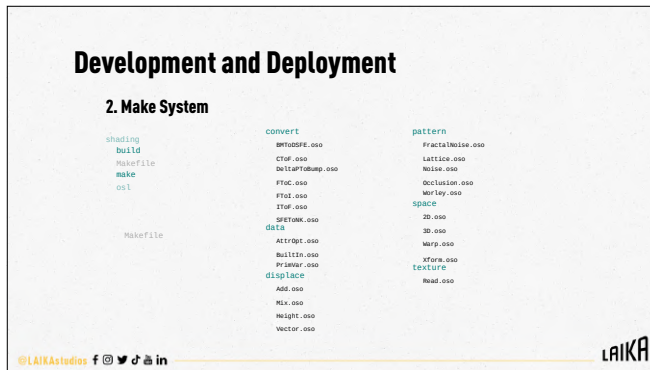
```
pattern
  FractalNoise.osl
  Lattice.oso
  Noise.oso
  Occlusion.oso
  Sierley.oso
space
  01.osl
  20.oso
  30.oso
  Warp.oso
  Warp.oso
  Warp.oso
texture
  Read.osl
  Read.oso
```

@LAIKAstudios f @ t d in

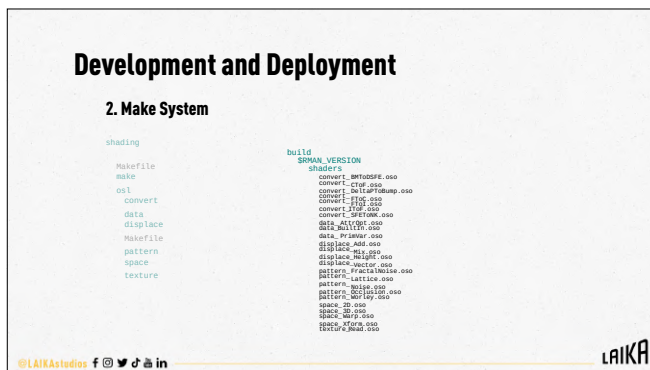
LAIKA

...compiling all the dot-osl files into dot-oso files. The Makefile detects when there's a change in a source osl file, and will selectively re-compile only that shader if it's newer than its corresponding oso file.

This makes osl shader development quick and easy by simply executing ‘make’ from within the osl directory during the development process.



Once the oso files have been built, a script from the top-level make directory deploys the complied shaders into the build directory so they can be used by RenderMan.



The script creates build sub-directories based on the desired RenderMan version, and renames the complied shader files to include their directory category. This makes them very easy to find in the RenderMan Bridge applications.

This also makes them usable in Universal Scene Description files, where only the shader's file name is used to specify them.

The whole system uses simple, standard Makefiles, a couple environment variables, and a basic python script to do the shader file renaming. It also runs in parallel, and can do multiple builds of the same shaders for any number of RenderMan versions.

AUTO

Development and Deployment

1. Source Organization
2. Make System
3. Debugging

@LAIKAstudios f @ t d in

LAIKA

Development and Deployment

3. Debugging
- it
 - printf()
 - rib file

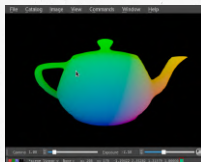
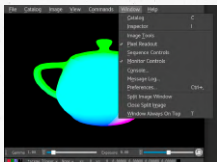
@LAIKAstudios f @ t d in

LAIKA

Since OSL shaders are executed as part of a larger rendering process, we can't use a typical code debugger to step through their execution or set break points. The only things we can control are what comes out of the shader, and what goes in.

Development and Deployment

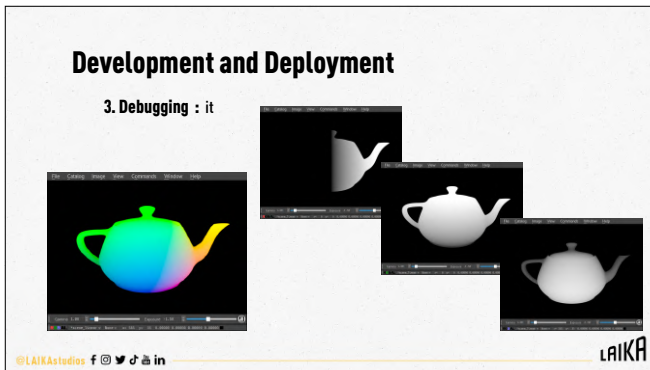
3. Debugging : it



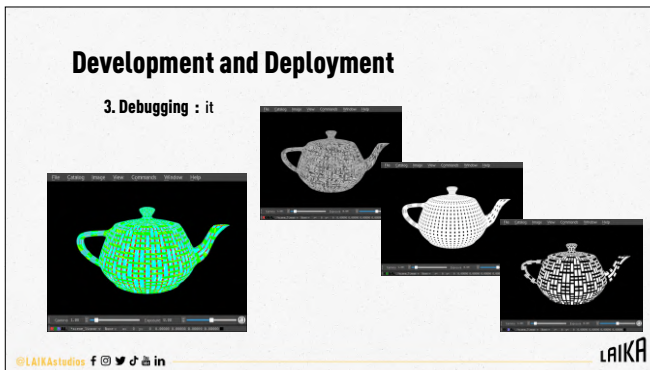
@LAIKAstudios f @ t d in

LAIKA

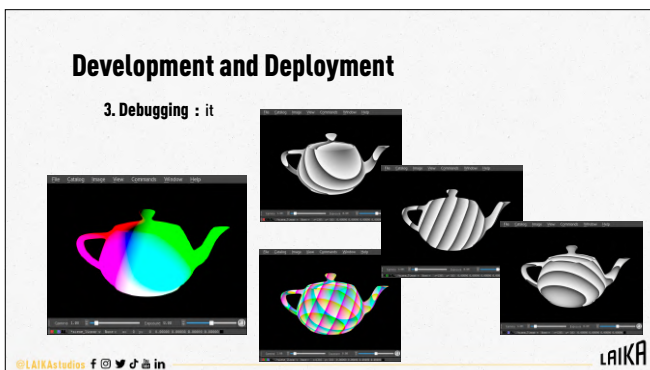
The primary output we have is the rendered image itself. Fortunately, the "it" image tool that comes with RenderMan has a Pixel Readout Window that tells you exactly, what RGB values, a pixel contains. The Monitor Controls also let you adjust the image to expose more of its variation without altering the pixel values.



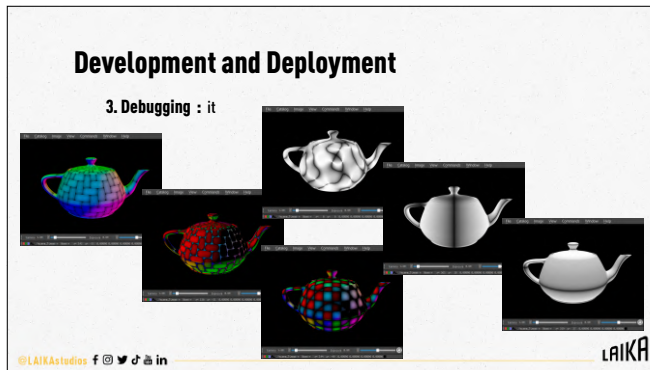
It also lets you view the red, green, and blue channels separately.



This can be quite useful, since you can set each color channel to a different float or integer variable's value to see how they relate to each other.

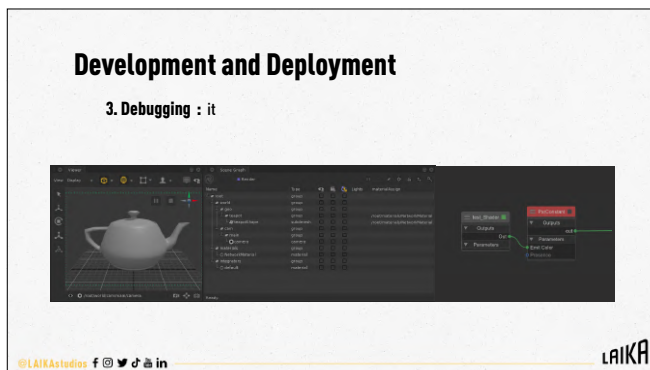


When a variable's range is large, sawtooth pattern generation can also be helpful to visualize its variation.



The `abs()`, or absolute value function, can also be useful for examining points, and the `dot()` product function can help you to understand what a vector or normal variable is doing.

The point is, writing shaders is about converting numbers into images, so use whatever creative abilities you have to produce an informative image of the variables you want to examine.

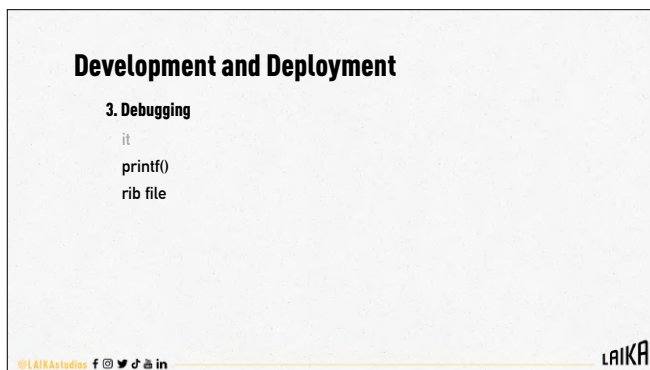


To render images for debugging purposes, you'll want to use the `PxrConstant bxdf`.

Connect your shader's color output to its Emit Color input.

Also, since the goal is to eliminate as many external factors as possible, it's helpful to render a scene that contains only your test object, with no lighting.

Not only will this render more quickly, but it will also ensure that the values you see in the image are exactly the values produced by your shader.

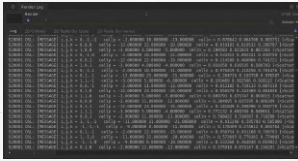


AUTO

Development and Deployment

3. Debugging : printf()

```
cellp = ofloor + vector(i, j, k);
cellp = modf(cellp, round(Frequency*Period));
celln = noise("cell", cellp);
printf("i,j,k = %d,%d,%d cellp = %f celln = %f", i, j, k, cellp, celln);
q = cellp + vector(1,0,0);
cellpoint = q / Frequency;
cellmap = q - vector(0,freq);
// generate the distance measurement.
q = vector(0,freq);
dist = length(vector(q));
```



@LAIKAstudios f @ t d a in

LAIKA

Sometimes there's just no way for a rendered image tell you what your shader is doing.

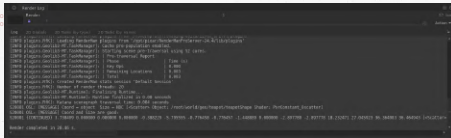
In those cases, OSL has a fairly standard printf() function that can be used to output any variable values you want.

Note that you can print tuple variable types with a single %f format specifier.

Development and Deployment

3. Debugging : printf()

```
{ if coord_is_good }
{
    matrix coord_to_size;
    int i,j,k;
    Out += (int) coord_to_size[0][0];
    printf("Coord = %d Size = %d", Coordinates, SizeSpace);
    printf("Coord and Size are good: %f", coord_to_size);
}
else
{
    Out += (int) coord_to_size[0][0];
    printf("Coord and Size are bad: %f", coord_to_size);
}
```



@LAIKAstudios f @ t d a in

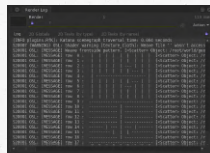
LAIKA

That includes matrix variables as well. This also makes it clear that a matrix variable in OSL is 4x4 and not 3x3.

Development and Deployment

3. Debugging : printf()

```
printf("Wavefront patterns.");
for(int i=0; i < waveData.waveData; ++i)
{
    string waveData = "";
    for(int j=0; j < waveData.waveData; ++j)
    {
        if(waveData[i][j] < 0)
        {
            waveData = concat(waveData, " ");
        }
        else
        {
            waveData = concat(waveData, " ");
        }
    }
    printf("row %d : %s", i, waveData);
}
```



@LAIKAstudios f @ t d a in

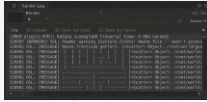
LAIKA

You can even use the ancient technique of printing a representative image with characters for especially difficult issues that can be represented as an array of values.

Development and Deployment

3. Debugging : printf()

```
printf("Scene framebuffer pattern:");  
for (int j=0; j < framebuffer.width; ++j)  
{  
    char framebuffer_w[j];  
    for (int i=0; i < framebuffer.height; ++i)  
    {  
        printf("%d, ", framebuffer[i][j]);  
        framebuffer = concat framebuffer, " ";  
    }  
    printf("%d", j, framebuffer);  
}
```



However, when using `printf()`, be aware that only unique outputs will be printed.

If your code prints the same result multiple times, you'll only see the first instance.

That's why, in the previous example, the row numbers were also included in the `printf()` statement. Without them, you'd see this set of messages instead.

Development and Deployment

3. Debugging

```
it  
printf()  
rib file
```

That covers the output side of debugging. Now for the input side.

Development and Deployment

3. Debugging : rib file

When developing shaders for RenderMan, it can be helpful to become familiar with the RenderMan Interface Bytestream, or “rib” file.

When viewed as ASCII text, a rib file will tell you every detail about the scene: what objects it contains, their shaders, the shader connections and parameter settings - any primvars or attributes or coordinate systems that are present - everything.

It literally *is* what the renderer is rendering.

The RenderMan Interface would be an entire course all on its own, so I'll just touch on some of the more pertinent things you can look for when debugging shading issues.

The first step is to save a debug rib file of your scene using your RenderMan Bridge Application.

The portion shown here includes everything up to the start of the scene description itself.

It contains information about where to look for shaders, textures and other resources; what Integrator you're rendering with, the camera Projection, and where the rendered image will be sent and what channels it contains.

[illegible]

LAIKA

After that comes the scene itself, which starts with `WorldBegin`.

Then comes an `AttributeBegin/End` block that describes a complete shading network.

It includes a Bxdf and a Displacement, along with all the Pattern shading nodes that are connected to them or each other.

[illegible]

LAIKA

Fixed parameter values are specified by their type and name, followed by the value in square brackets.

[illegible]

LAIKA

Every "reference" you see defines a connection between shading nodes. It specifies the shader parameter that's being connected to. In the square brackets after that is the name of the shading node and its output parameter that's connected to it.

In the square brackets after that is the name of the shading node and its output parameter that's connected to it.

In the square brackets after that is the name of the shading node and its output parameter that's connected to it.

LAIKA

Then comes an `ObjectBegin` declaration that defines an object. Its name is specified by the Attribute `"identifier"` string. In this case, it's called `"teapotShape"`.

This geometry's definition also includes the previously defined displacement shader.

This geometry's definition also includes the previously defined displacement shader.

LAIKA

it also has an "st" primitive variable.

LAIKA

[illegible]

It contains a couple of user-defined coordinate systems, the object's group membership,

[illegible]

Then come the user-defined coordinate systems, which are also available to light sources,

followed by an instance of the light itself.

RenderMan OSL

Code Examples

Interface Design

Development and Deployment

[@LAIKAsudios](#) [f](#) [@](#) [t](#) [v](#) [d](#) [a](#) [i](#) [n](#)

LAIKA

Development and Deployment

And to the end of this course.



Laika generously contributed my time to its development, so I'd like to thank all those at Laika who made it possible.

Laika is also committed to open source software, and has a number of open source repositories on GitHub;

one of which contains the supplemental materials for this course.

In it you'll find all the OSL source files and the makefile system used to develop and deploy them.

Please feel free to download or clone it.



Thank you so much.